Machine Learning for Neuroscience

Prediction and Learning

BioRTC

UNIVERSITY OF SUSSEX

sussex neuroscience

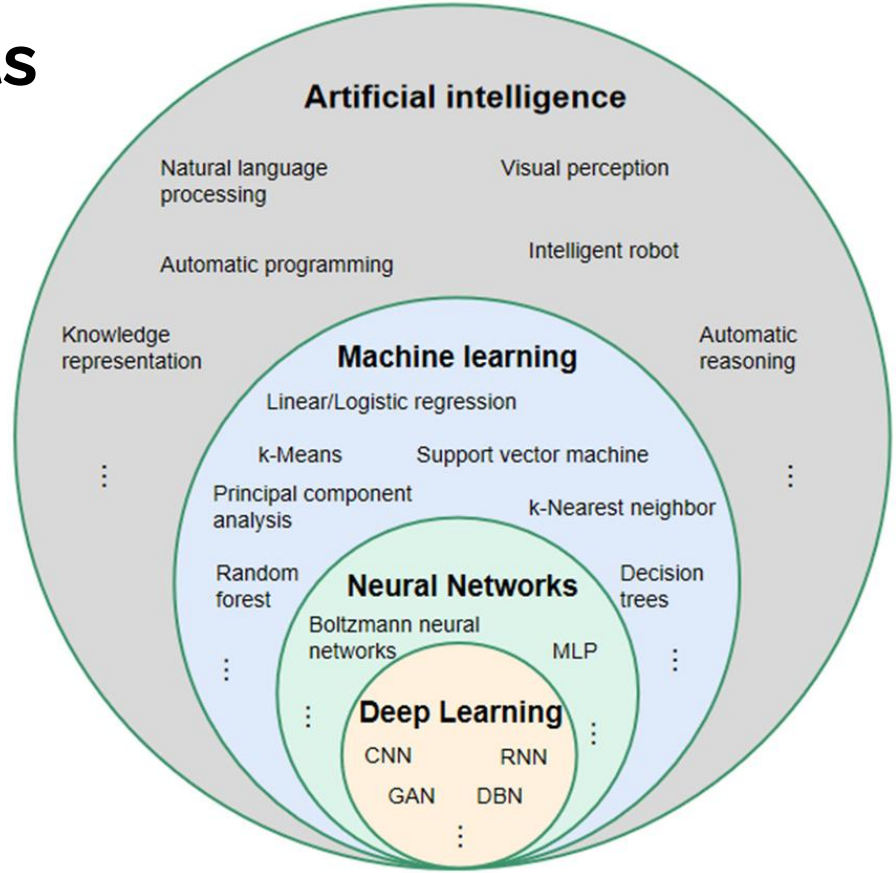
I ♥ Maravall Lab

Sussex AI

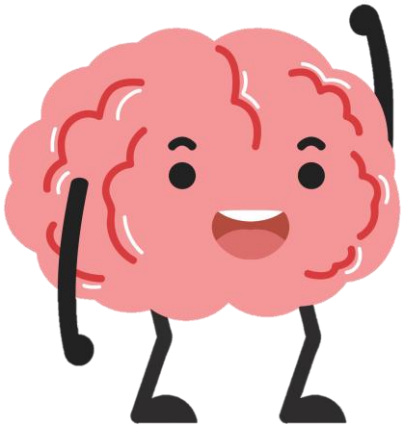
Recap

- We have looked at different datasets (calcium imaging, DANDI, etc…)
- We learned about the different datatypes (Boolean, integer, float, string)
- We learned the different types of dimensionality reduction (PCA, T-SNE, UMAP)
- We learned about VAEs

Contents


Artificial intelligence
Natural language processing
Visual perception
Automatic programming
Intelligent robot
Knowledge representation
Automatic reasoning
Machine learning
Linear/Logistic regression
k-Means
Support vector machine
Principal component analysis
k-Nearest neighbor
Random forest
Neural Networks
Decision trees
Boltzmann neural networks
MLP
Deep Learning
CNN
RNN
GAN
DBN

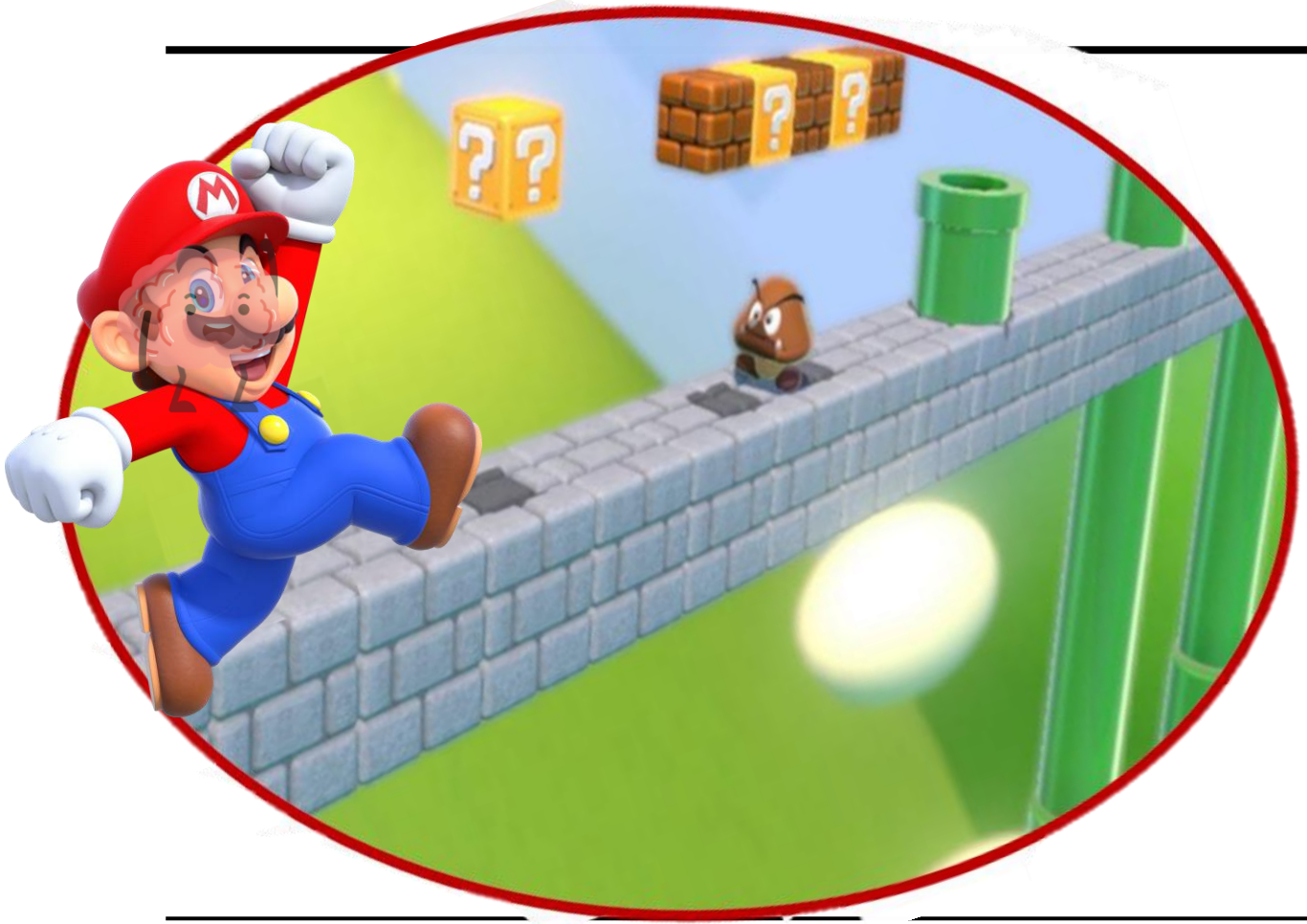
Why prediction is important for us



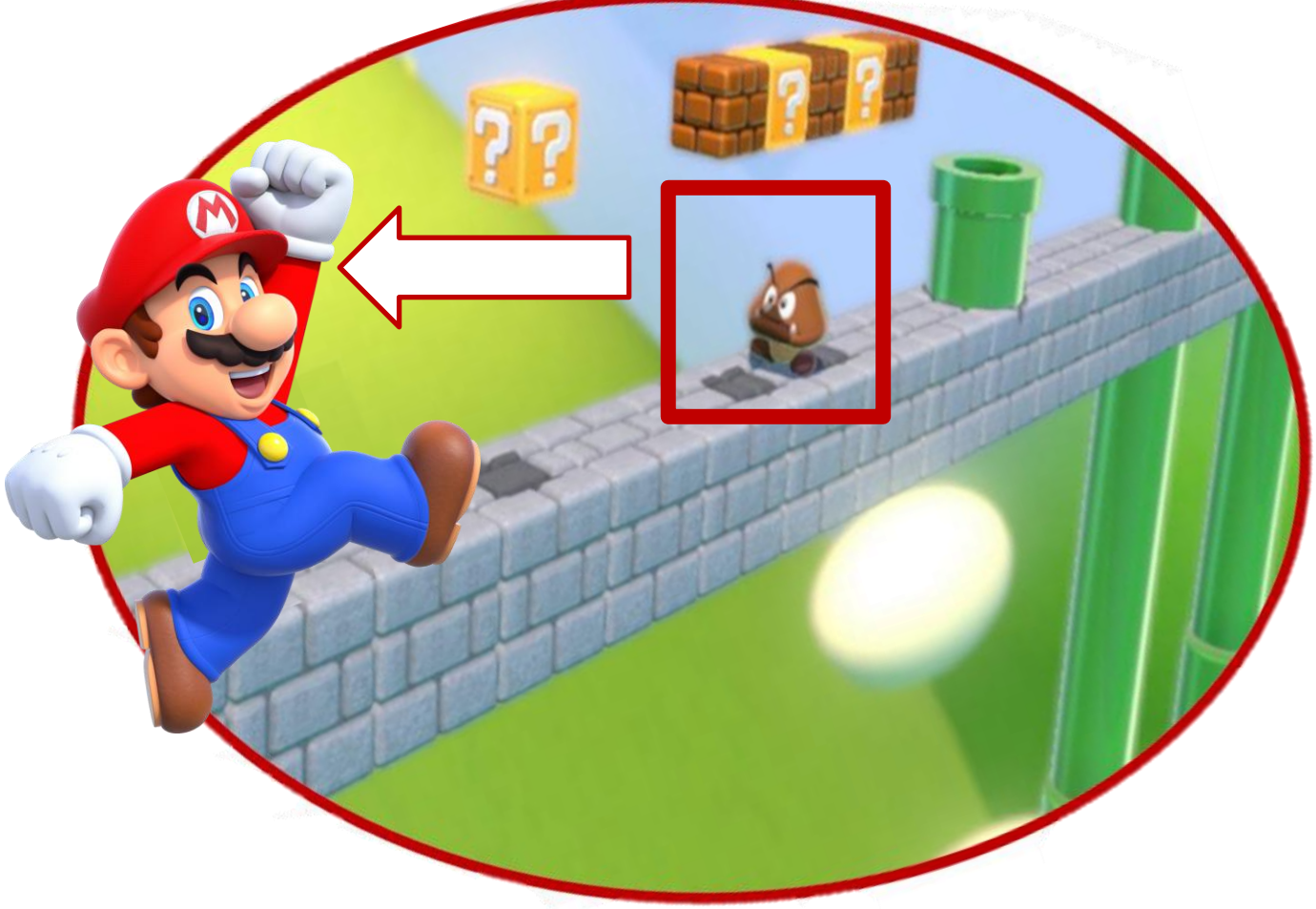
We have a tendency to just think of the brain in isolation

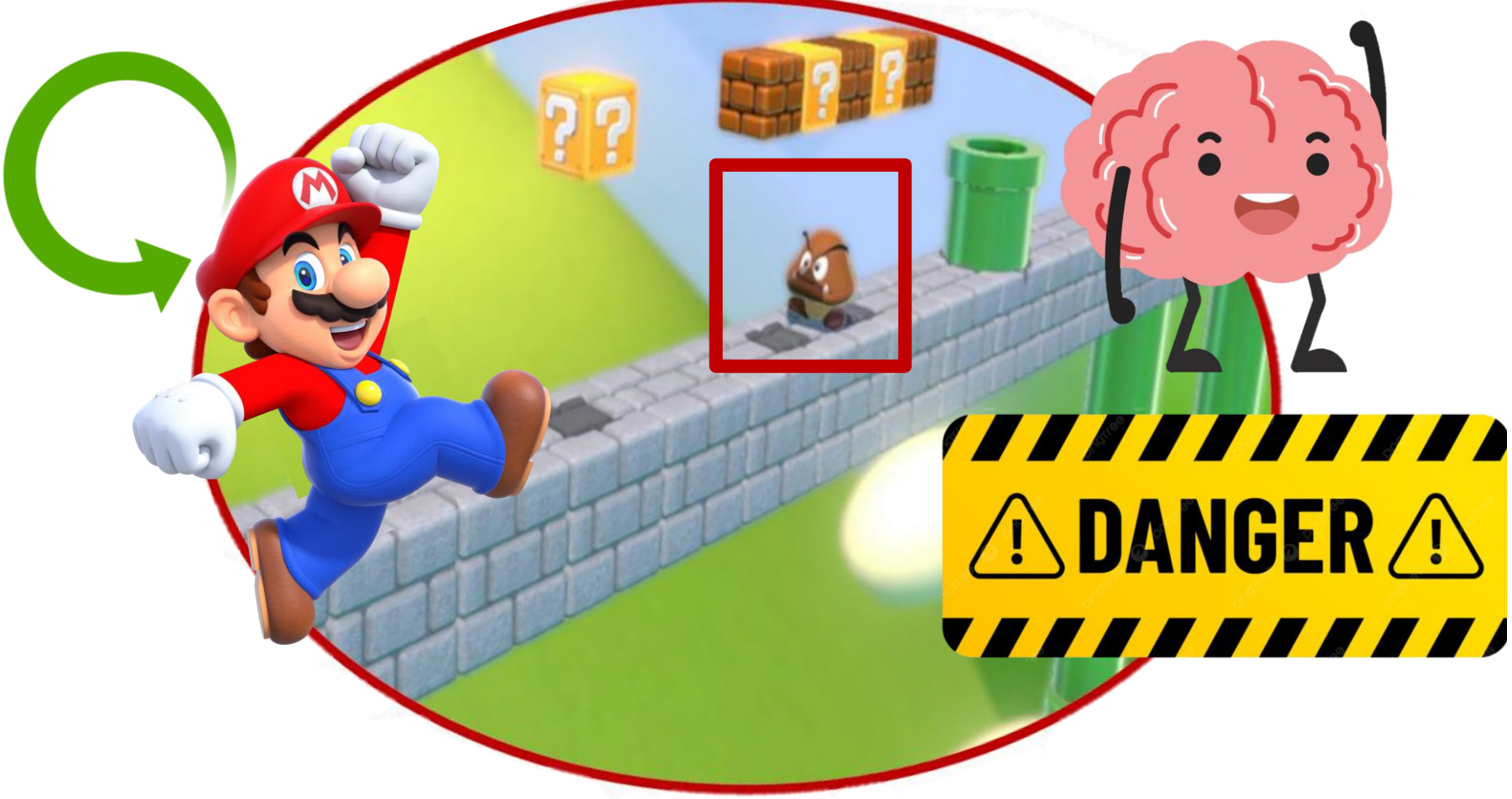
Why prediction is important for us

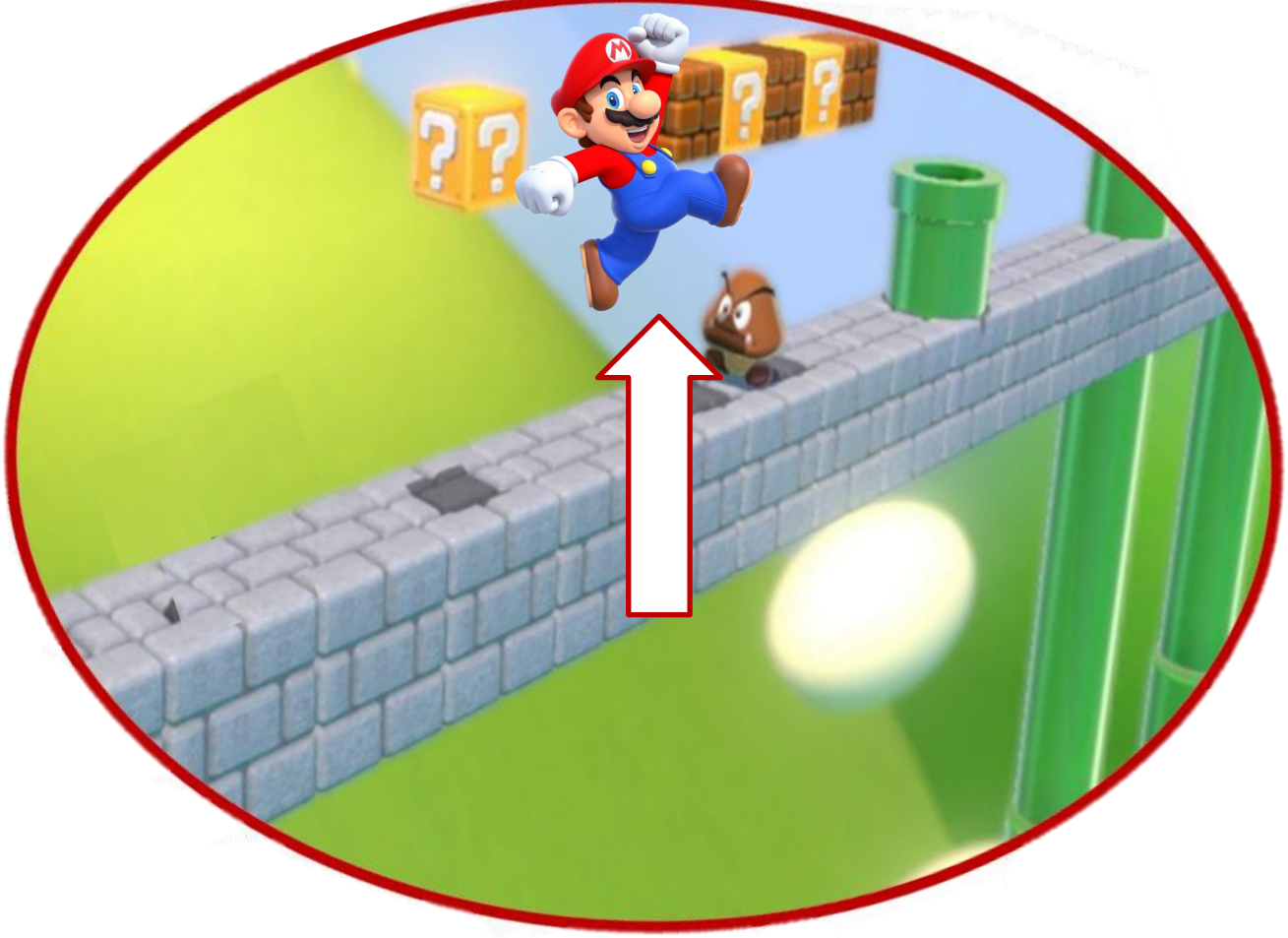
But the brain is embodied



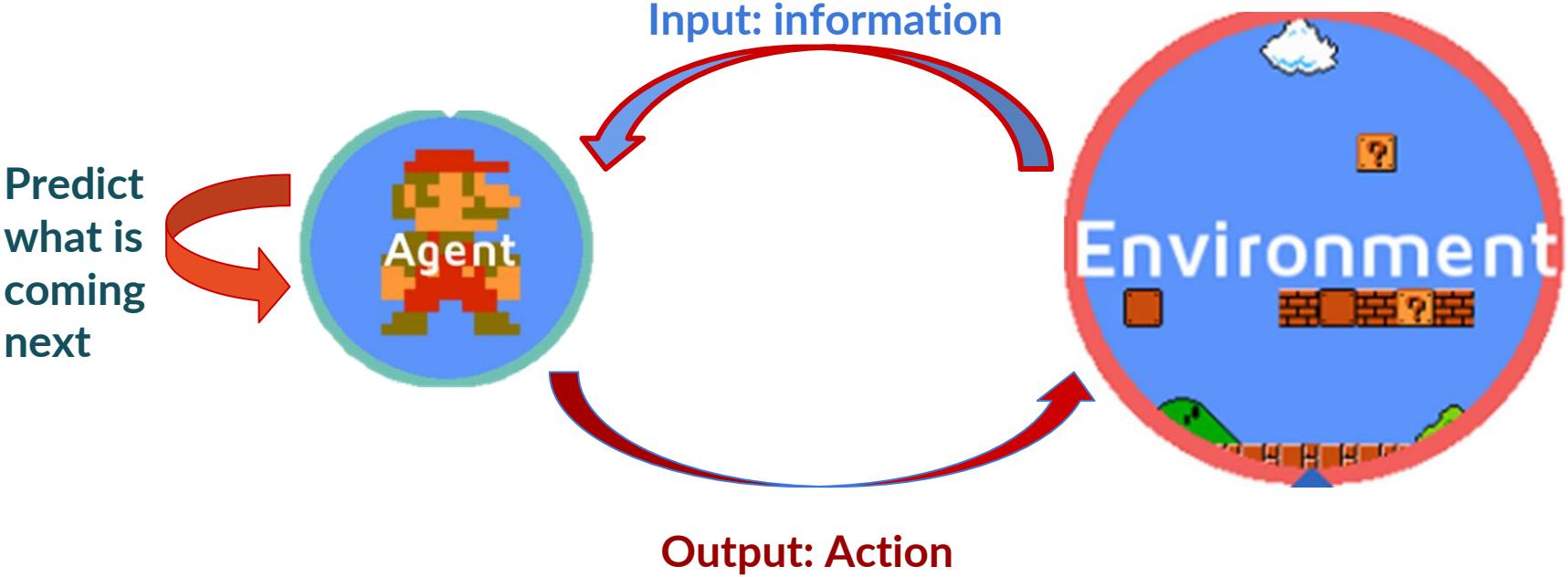
And embedded in an environment



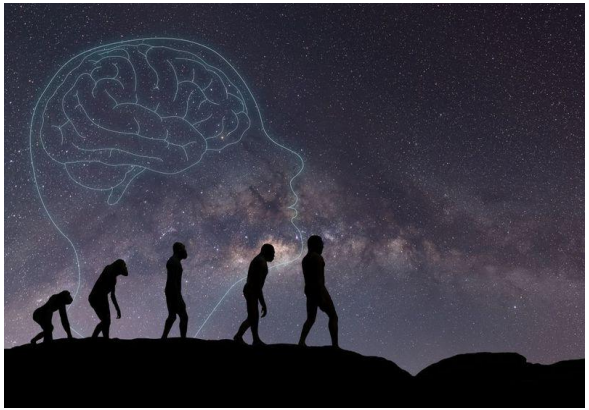

DANGER



Why prediction is important for us



The brain is designed for pattern recognition



Pattern recognition helped our ancestors survive by allowing them to **predict** and respond to environmental changes, such as *weather patterns* or *animal behaviours*:

- Dark clouds + intense wind ⇒ very likely rain ⇒ seek repair

- Tiger growling and assuming an aggressive stance ⇒ likely going to attack ⇒ run away

UNIVERSITY OF SUSSEX

Patterns are how we make sense of the world

Patterns are all around us and we recognise them, whether explicitly or implicitly.

We recognise facial patterns in objects

We recognise patterns in music (even if we cannot really recognise what they are exactly)

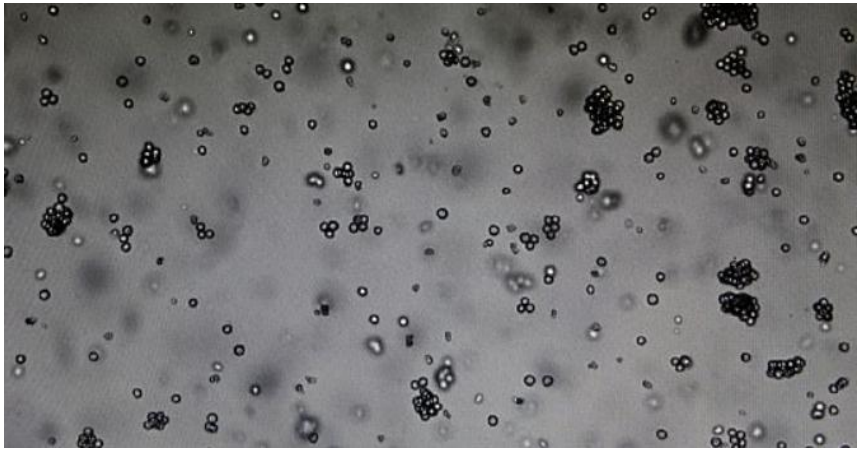
Pattern recognition is crucial for prediction

Computers can identify patterns that we cannot identify

Enters the digital age

AI designed to distinguish croissants from crullers and other pastries proves capable of identifying cancerous cells on microscope slides with 99% accuracy

- BakeryScan is designed to recognize different pastries to assist at bakeries
- It is equipped with deep learning for object recognition to complete the task
- However, the system was tested in hospitals to see if it can spot cancerous cells
- The system scans a microscope slide, identifies cells and measures the nucleus



The algorithm used to recognise the types of bakery items on a tray was repurposed to recognise types of malignant cancerous cells from the benignant

UNIVERSITY OF SUSSEX

Imagine all the things we can achieve in neuroscience once we master such powerful tools

Types of learning

Unsupervised

Algorithms learn patterns exclusively from unlabelled

Used for specific algorithms such as clustering, dimensionality reduction, and association

Supervised

You have data that is labelled

Typically used in most ML problems

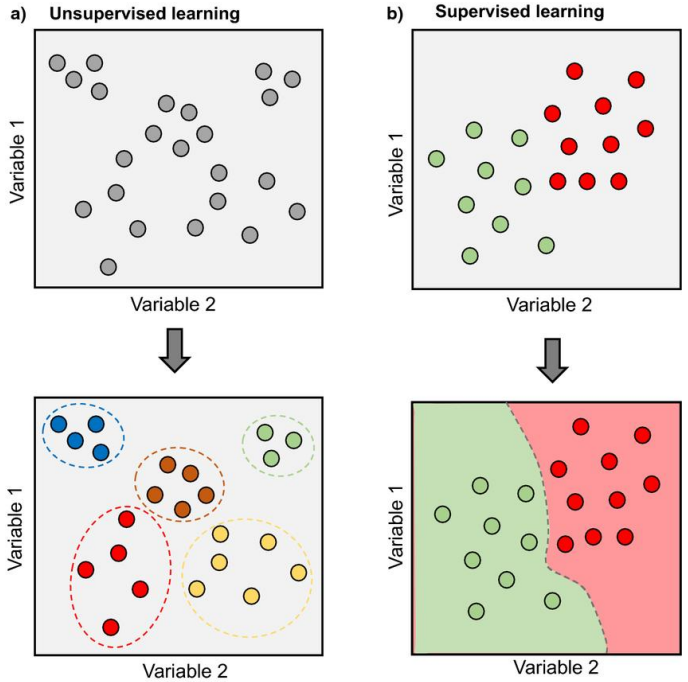
Requires gathering of large datasets

Types of learning

Unsupervised

Algorithms learn patterns exclusively from unlabelled

Used for specific algorithms such as clustering, dimensionality reduction, and association



Supervised

You have data that is labelled

Typically used in most ML problems

Requires gathering of large datasets

UNIVERSITY OF SUSSEX

Supervised example

We have our data (known as X)

Supervised example

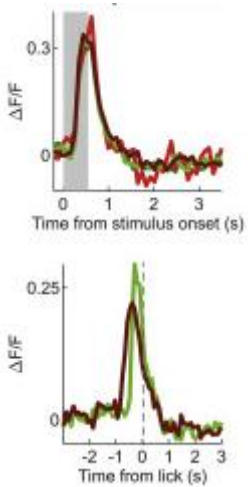
We have our labels (known as y)

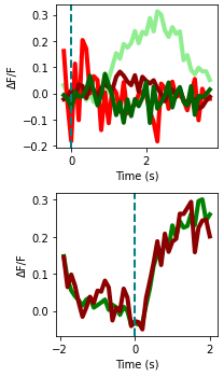
y= Class=cat

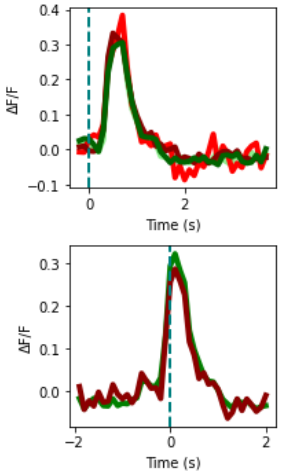
y= Class=dog

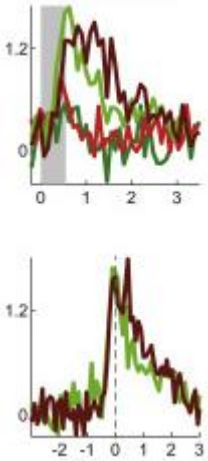
Supervised example

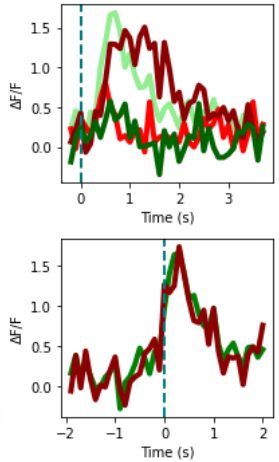
Our data (X)

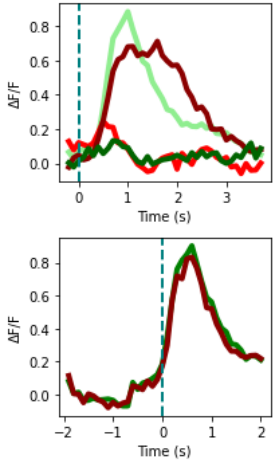








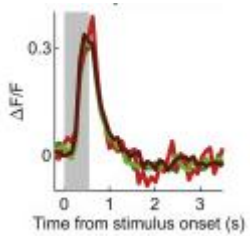


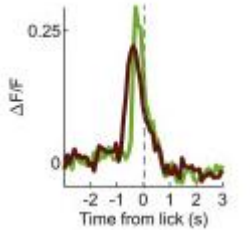


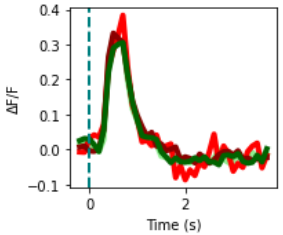
UNIVERSITY OF SUSSEX

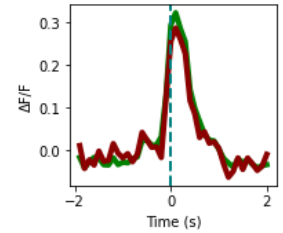
Supervised example

Our labels (y)

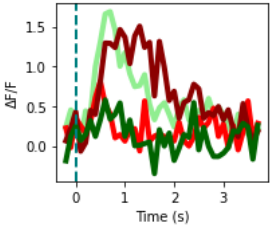


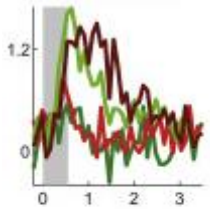


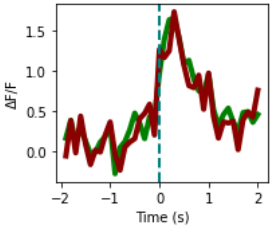


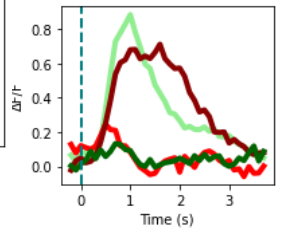


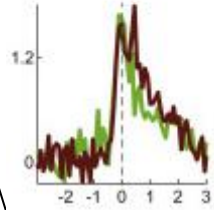
y = Class=sensory cue/ touch

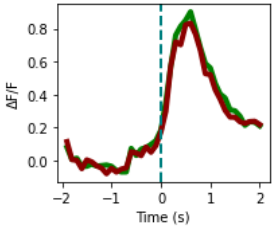












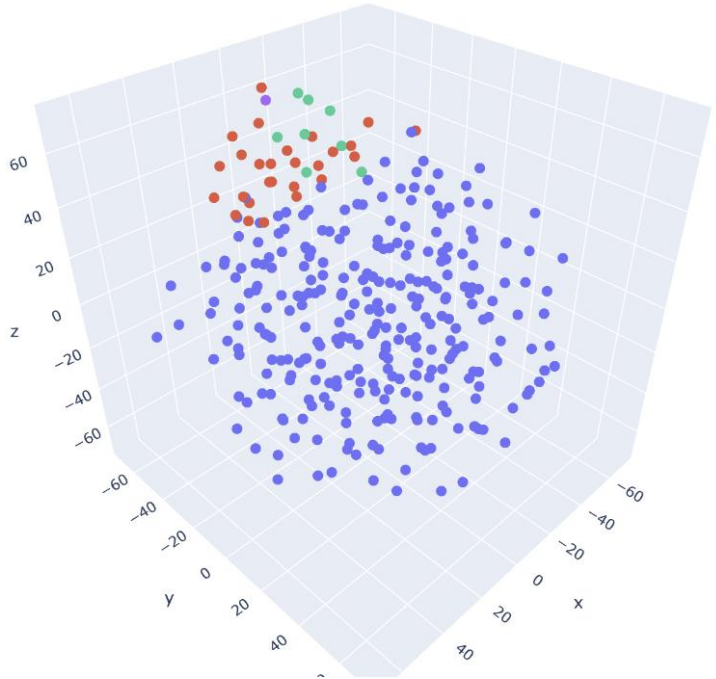
y= Class= decision / action

UNIVERSITY OF SUSSEX

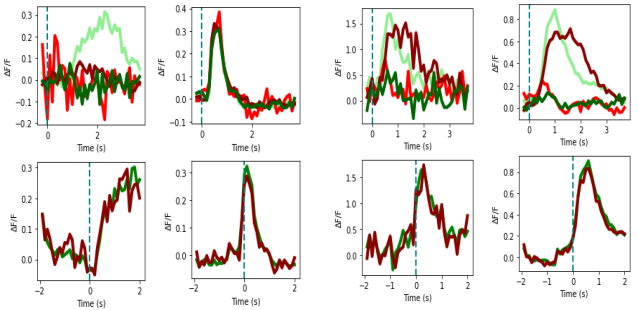
Un-supervised example

Using a clustering method known as **k-means** we can clearly segment the data into classes

This method does not need labelling



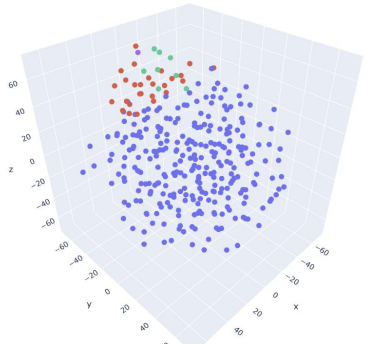
Supervised vs unsupervised



Labelling data costs time (and money)

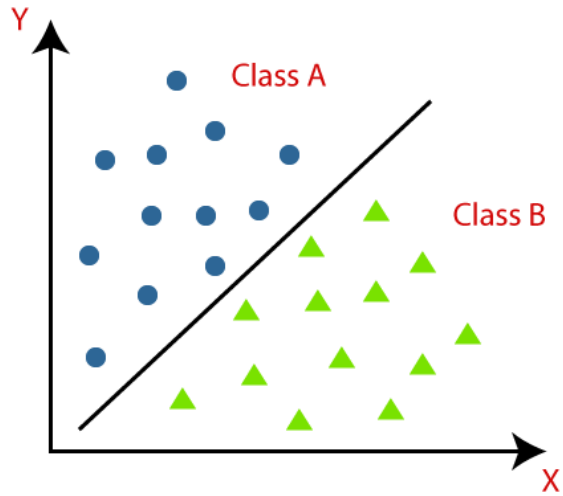
Data needs to be high quality and relevant for unsupervised

Unsupervised is helpful when data gathering is challenging eg. labelling something you don't always know.



Overview of Classification

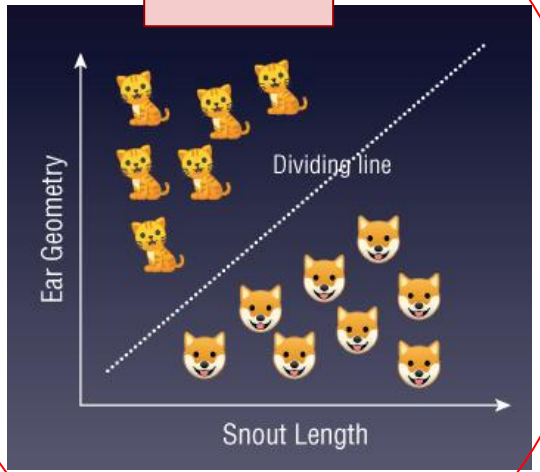
Classification is is a machine learning technique used to **categorize data into distinct groups or classes**. It helps us assign labels to new data points based on **patterns** learned from past data.



Binary Classification

Goal: find decision boundary *(dividing line between classes)*

SVM

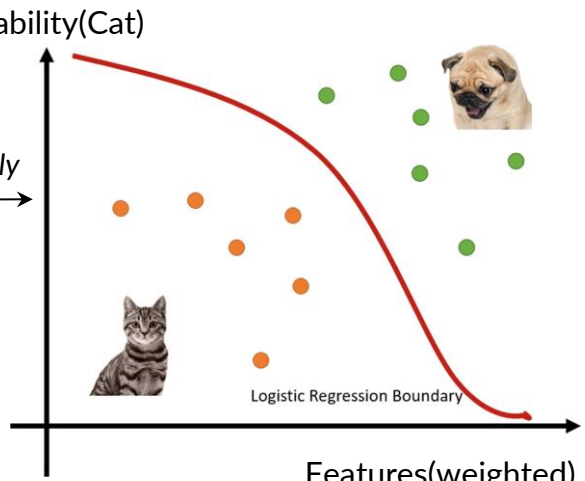


Geometrically

Cat or Dog?

Probabilistically

Logistic Regression

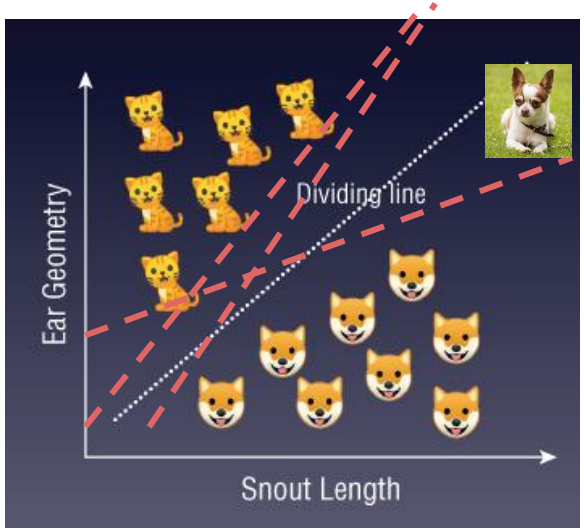


UNIVERSITY OF SUSSEX

Binary Classification - SVM

Goal: find **lower-dimensional** decision boundary - hyperplanes

-> Separates classes in the feature space



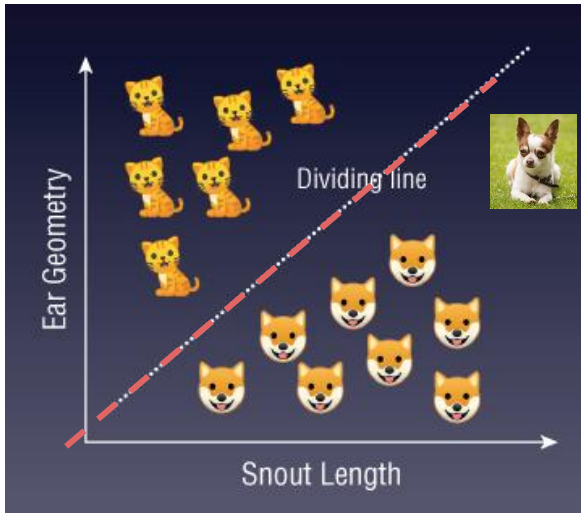
-> Cat X

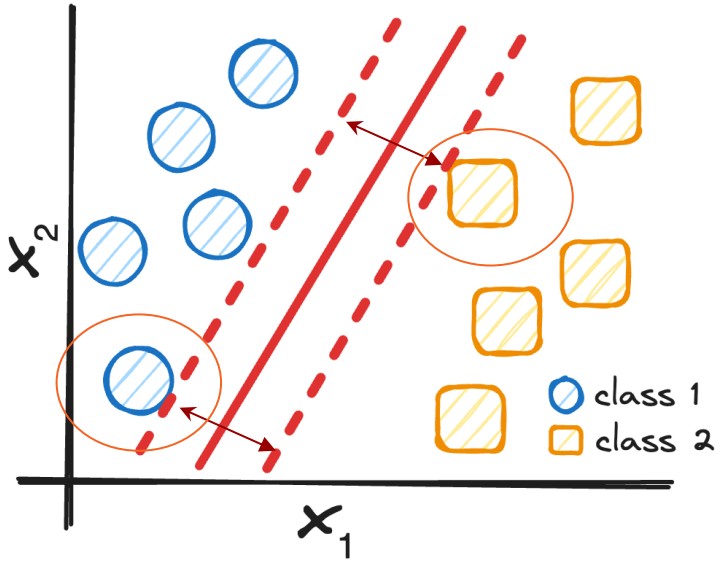
Cat or Dog?

Support Vector Machine (SVM)

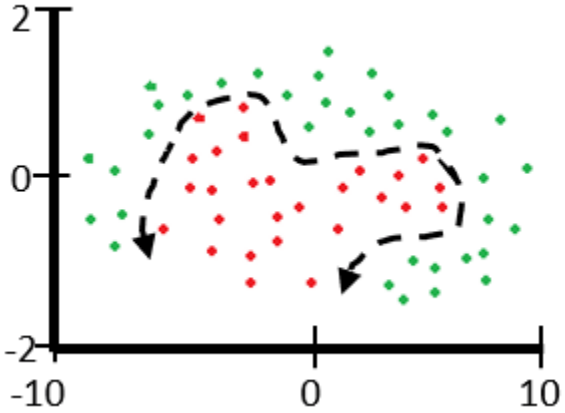
Goal: find **hyperplane** that **maximizes the margin between the support vectors**

$w \cdot x + b = 0$ *distance* *nearest points of each class*





What if we have non-linearly separable data?



-> no optimal hyperplane possible!

Standard SVM fails when data is not linearly separable.

Solutions:

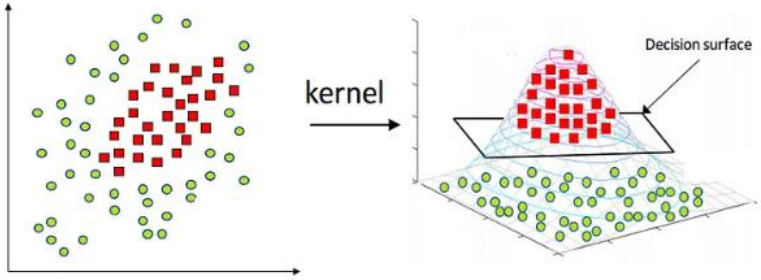
- **Soft Margin SVM**: allows misclassification by changing constrain function
- **Kernel Trick:** maps data into higher dimensional space

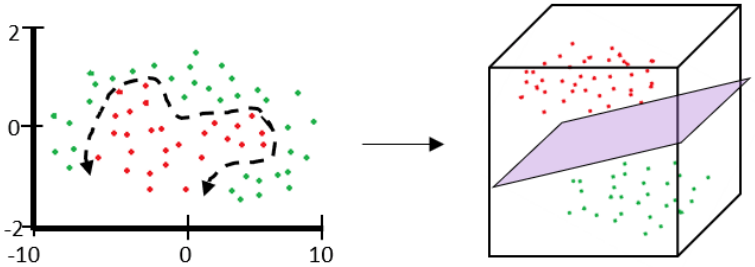
Kernel Functions

Use a kernel function to transform data into a higher-dimensional space before finding hyperplane.

Common Kernel Functions:

- **Linear**: no transformation, standard SVM
- **Polynomial**: curved decision boundaries
- **RBF (Gaussian), Sigmoid:** non-linear data, complex patterns





UNIVERSITY OF SUSSEX

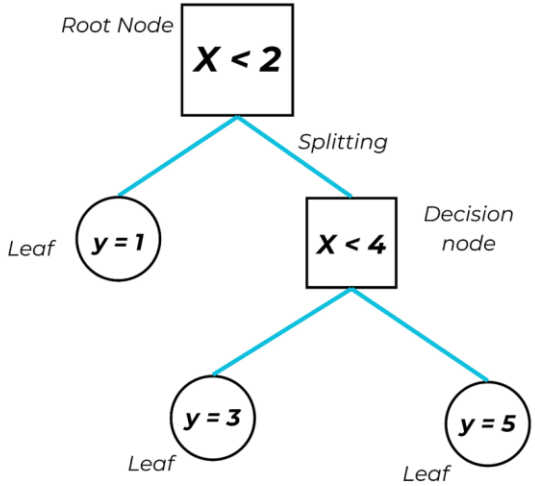
More complex classification problems

Random Forest

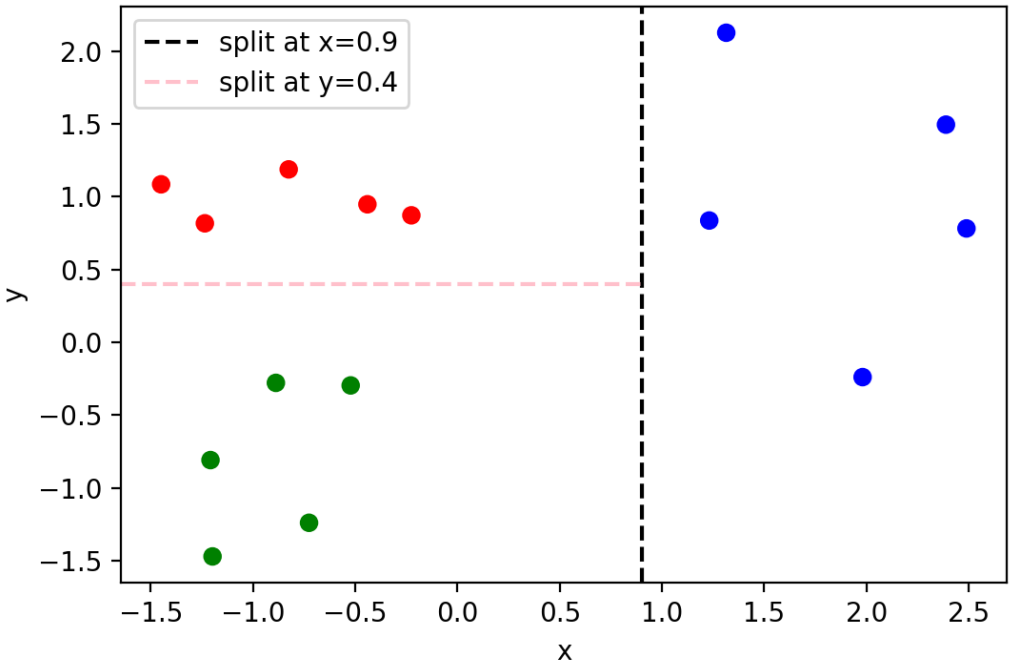
Decision Trees

Learns patterns in data by repeatedly splitting based on features, leading to a decision boundary that classifies the input data.

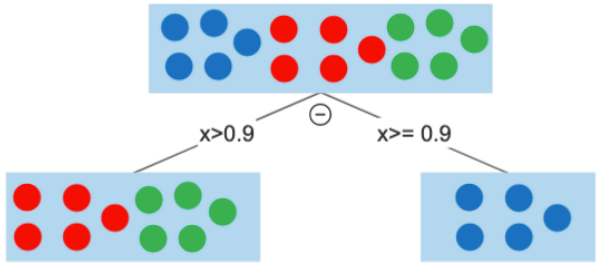
- **Root Node:** Starting point
- **Decision Nodes:** Split data based on features
- **Leaf Nodes:** Final nodes with predictions
- **Output:** Class label at the leaf node



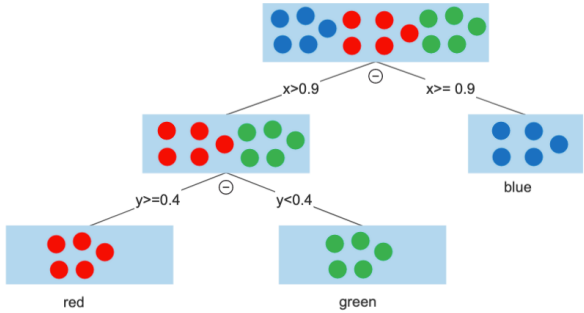
Example



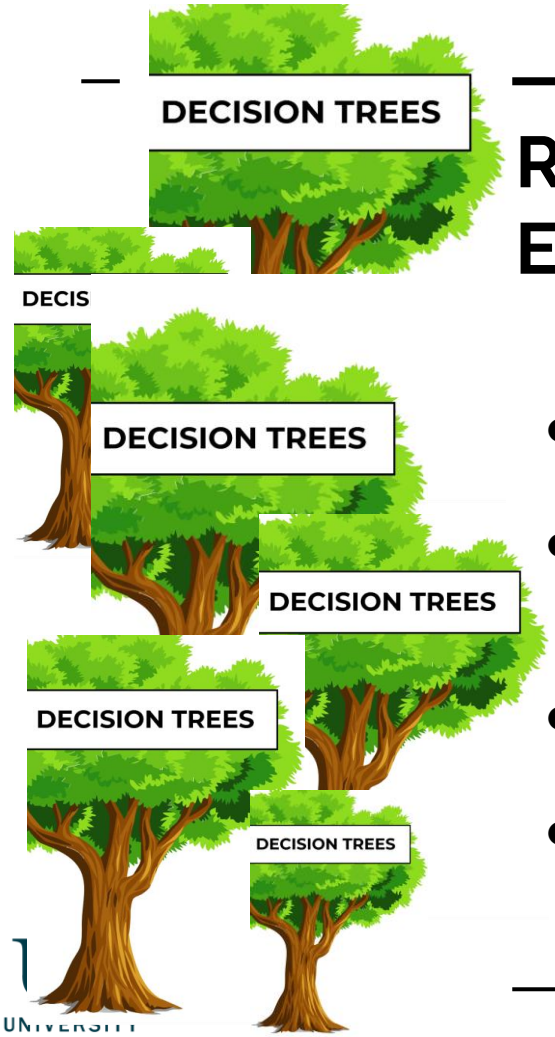
1.



2.



UNIVERSITY OF SUSSEX



Random Forest: Ensemble of decision trees

- A **collection ("forest") of Decision Trees**.
- Each tree is trained on a **random subset of the data** and **a random subset of features**.
- The forest **aggregates the output** of all trees
- For classification: takes the **majority vote**.

Splitting this data

Data splitting allows us to build robust models that perform well on training data and generalise effectively to new, unseen data.

Training data

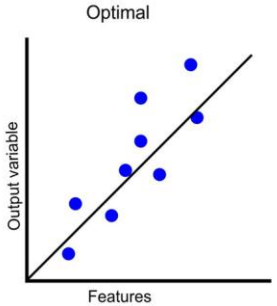
This is the section of the data set we use to train the model, our model might get really good at predicting this, but that does not mean the model works well

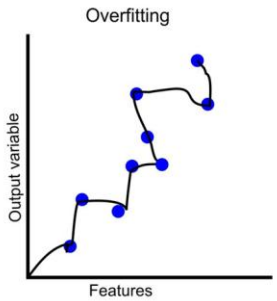
Testing data

The data we use to test how the model performs on unseen data. This gives a good idea of model robustness.

Overfitting

Overfitting occurs when a machine learning model learns the training data too well, capturing noise or random fluctuations in the data rather than the underlying patterns.



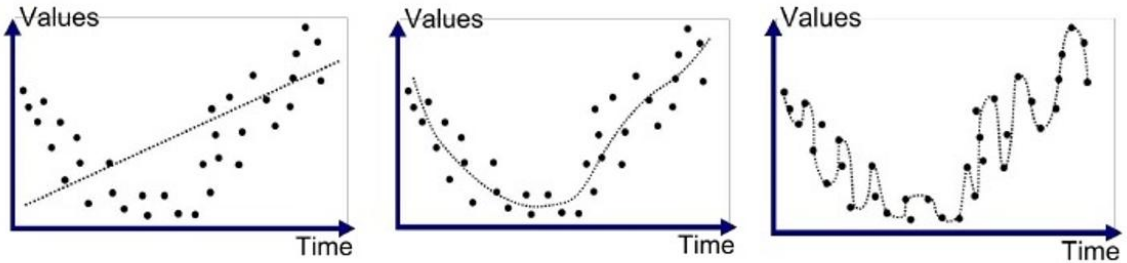


UNIVERSITY OF SUSSEX

Under fitting

Model has not been trained enough

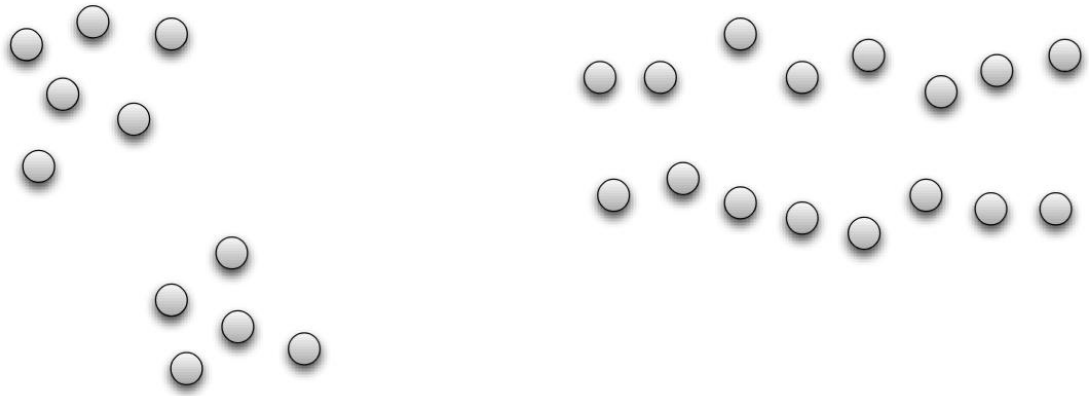
Data is not meaningful enough



Underfitted | Good Fit/Robust | Overfitted

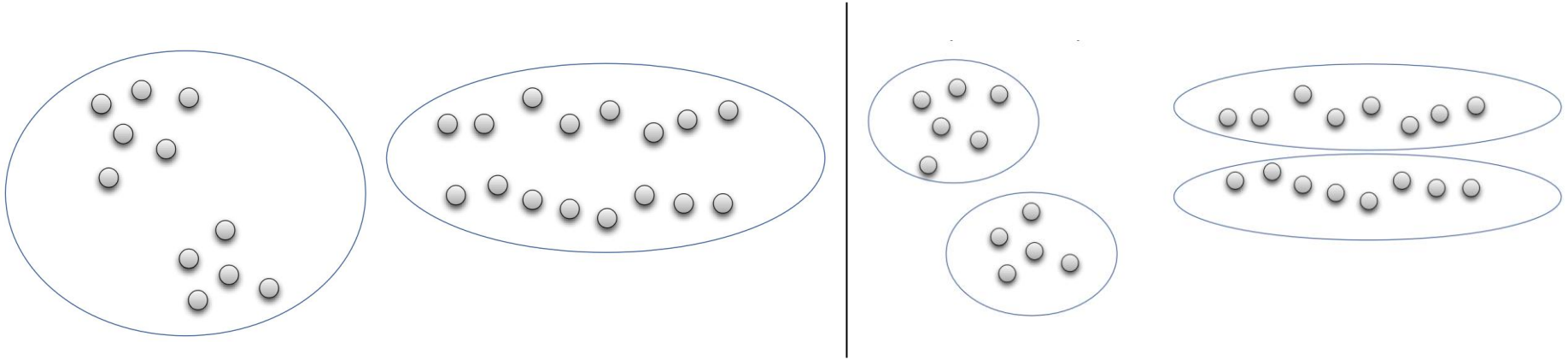
Clustering

- Basic idea: group together similar instances



Clustering

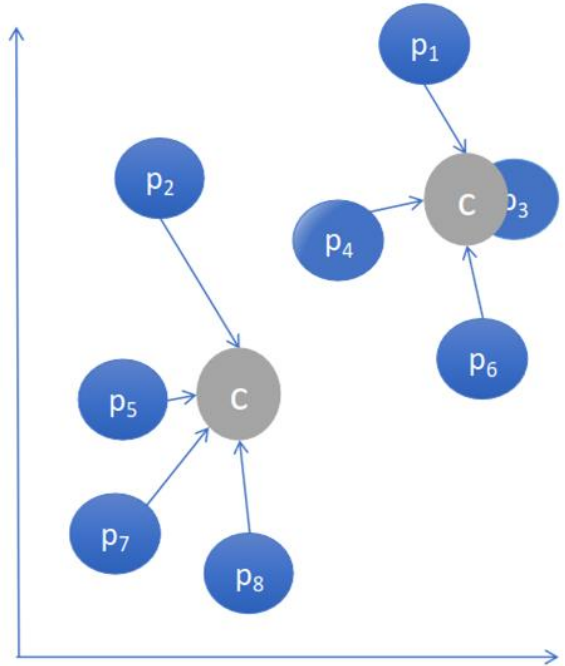
- Basic idea: group together similar instances



-> dependent on the measure of similarity (or distance) between “points” to be clustered

K-means Algorithm

- **Unsupervised**: No labels or predefined categories.
- **Clusters**: Groups of points that are more similar to each other than to those in other clusters.
- **Centroid**: The "centre" of a cluster, calculated as the mean of all points in the cluster.



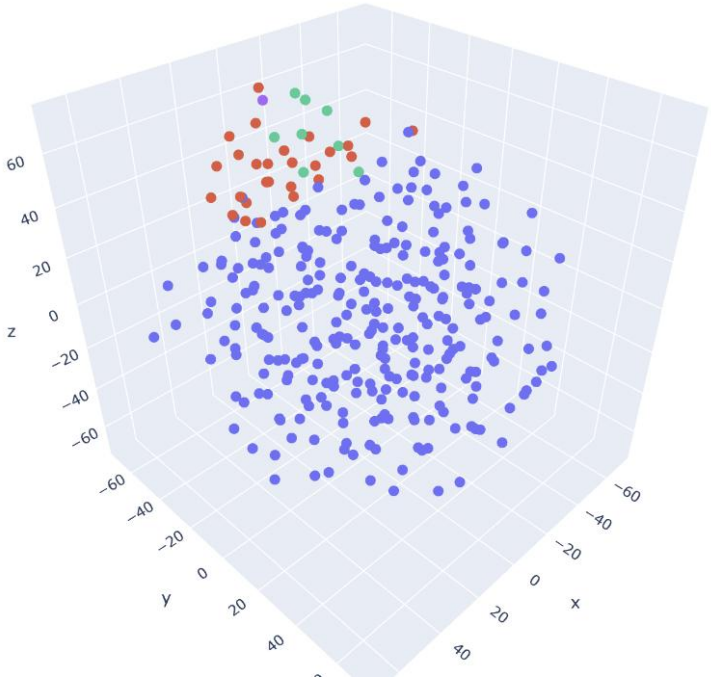
K-means Algorithm

In neuroscience, this is ideal for:

Grouping neurons with similar activity patterns;

Identifying states in brain imaging data;

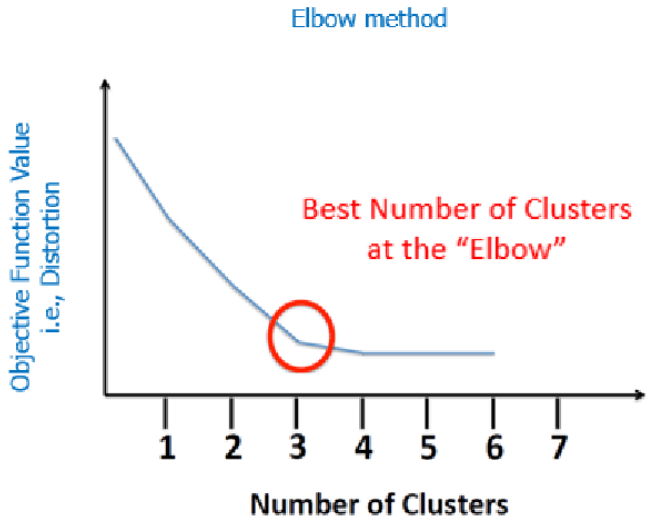
Segmenting behavioral responses in trial data



UNIVERSITY OF SUSSEX

K-Means

How many clusters to choose?



Neural Networks

BioRTC

UNIVERSITY OF SUSSEX

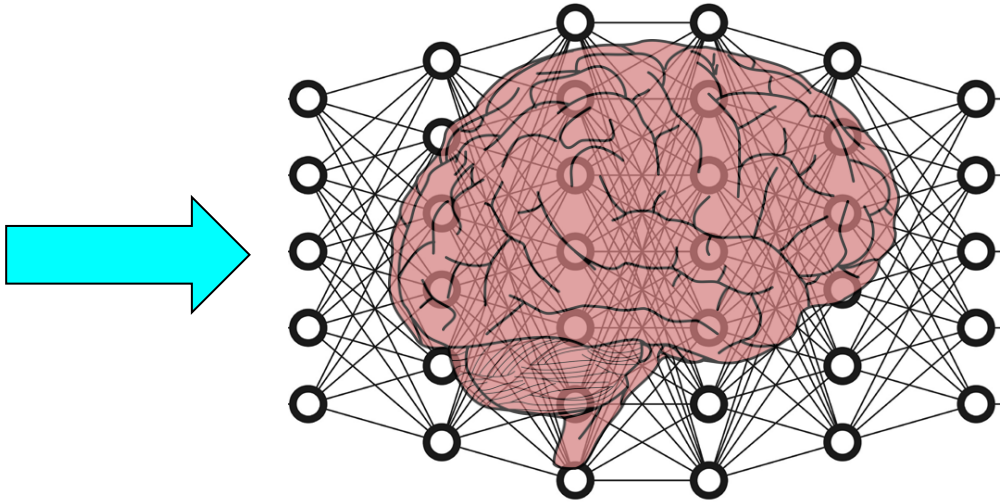
sussex neuroscience

I ♥ Maravall Lab

Sussex AI

Neurons and Artificial Neural Networks

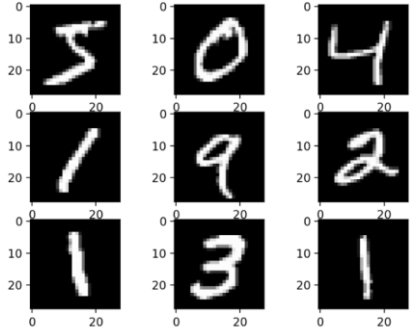
X: inputs



y: outputs

Artificial Neural Networks can be classifiers

Dataset

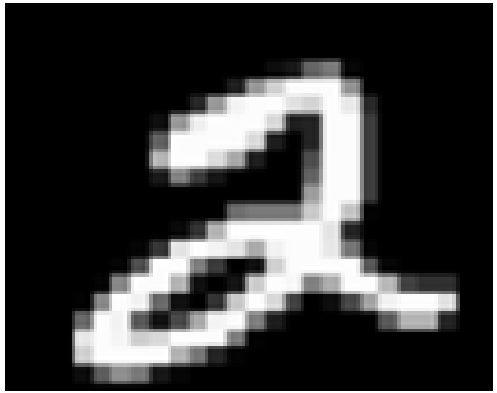


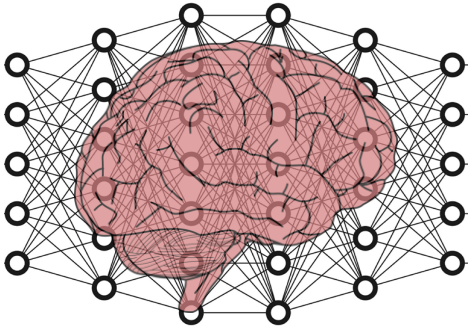
60,000 small square 28×28 pixel grayscale **images of handwritten single digits between 0 and 9**

Artificial Neural Networks can be classifiers

Image recognition

X:



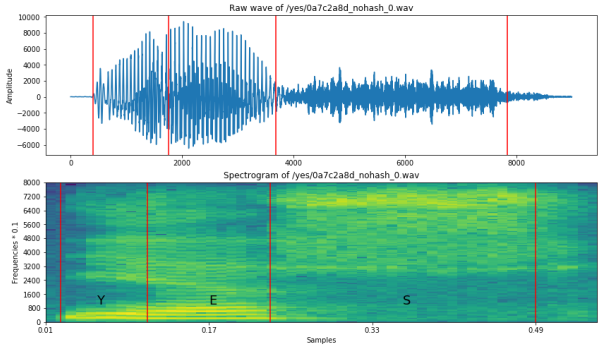


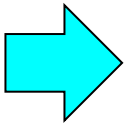
y: 2

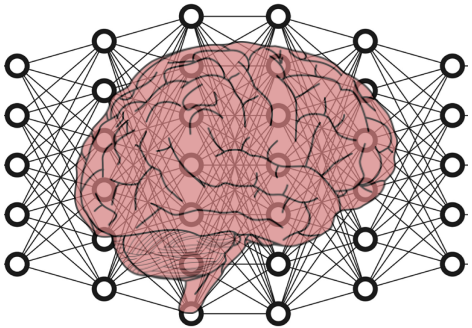
Artificial Neural Networks can be classifiers

Speech recognition

X:







y: “yes”

Spectrogram of the spoken word “yes”

Artificial Neural Networks

Can do countless things, including :

- **Natural Language Processing (NLP):**
  Input: Text data (e.g., a sentence or document).
  Output: Processed text (e.g., sentiment score, translated text, or named entities).

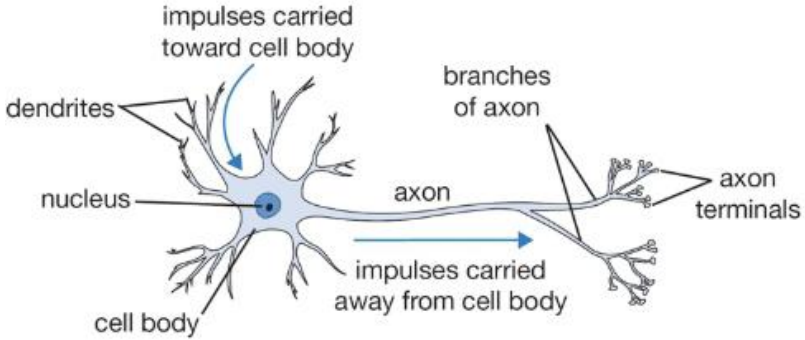
- **Medical Diagnosis:**
  Input: Medical images (e.g., X-rays, MRIs) or patient data (e.g., age, symptoms).
  Output: Diagnosis or probability of a condition (e.g., presence of a tumor).

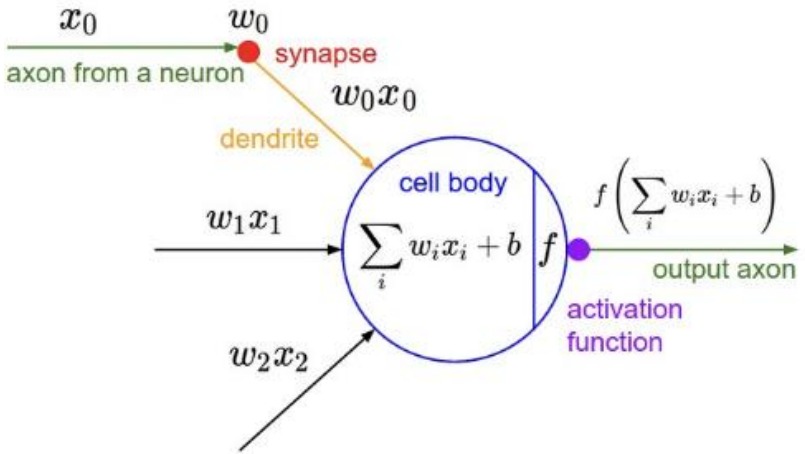
- **Financial Services:**
  Input: Historical financial data (e.g., stock prices, transaction records).
  Output: Predictions (e.g., future stock prices, fraud detection alerts).

- **Autonomous Vehicles:**
  Input: Sensor data (e.g., camera images, LIDAR scans).
  Output: Driving actions (e.g., steering angle, acceleration, braking).

- **Recommendation Systems:**
  Input: User behaviour data (e.g., past purchases, viewing history).
  Output: Recommendations (e.g., movies, products

Biological vs Artificial neuron

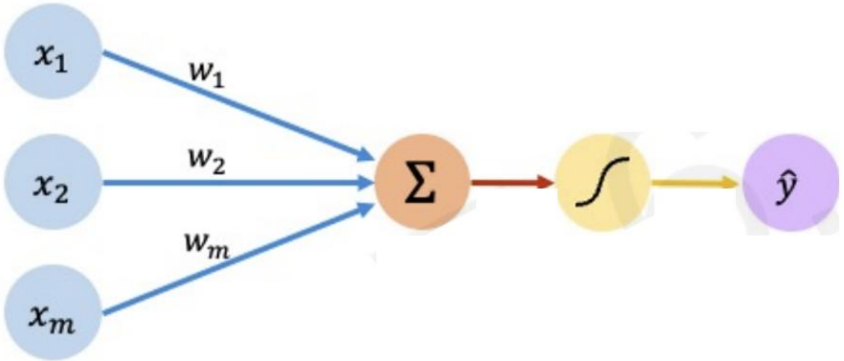


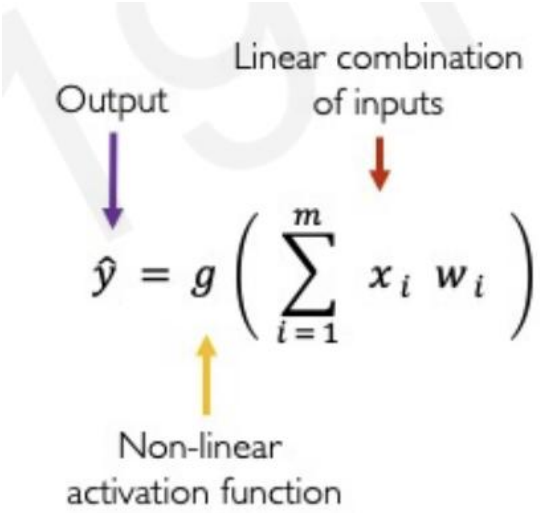


UNIVERSITY OF SUSSEX

Perceptron: the singular neuron

The building block of Artificial Neural Networks

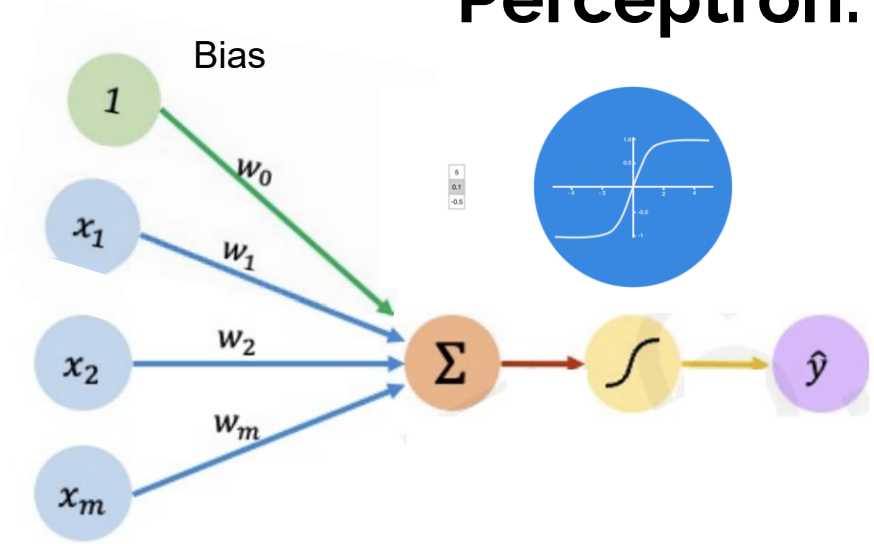


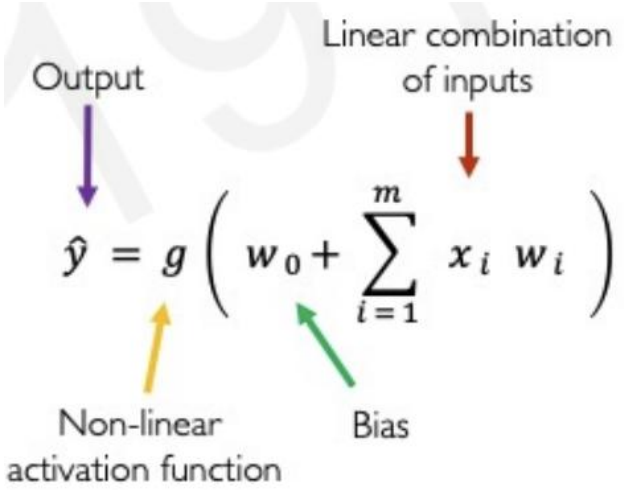


Inputs Weights Sum Non-Linearity Output

UNIVERSITY OF SUSSEX

Perceptron: the singular neuron

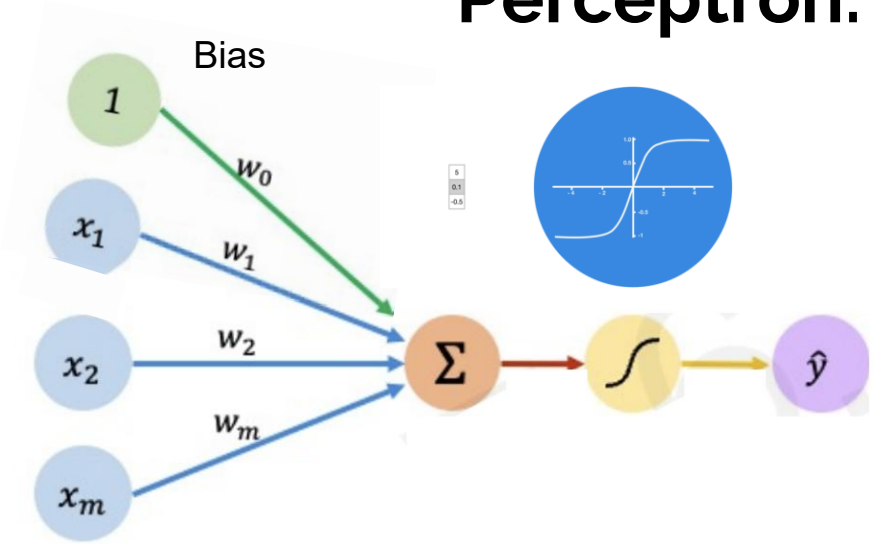




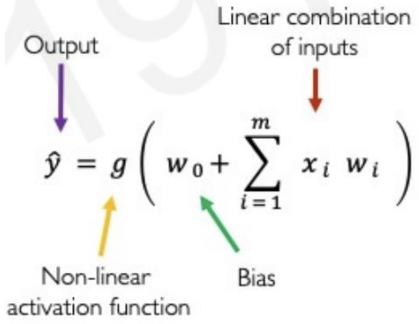
Inputs Weights Sum Non-Linearity Output

The bias term is a scalar that allows us to shift left or right along our activation function

Perceptron: the singular neuron



Inputs Weights Sum Non-Linearity Output



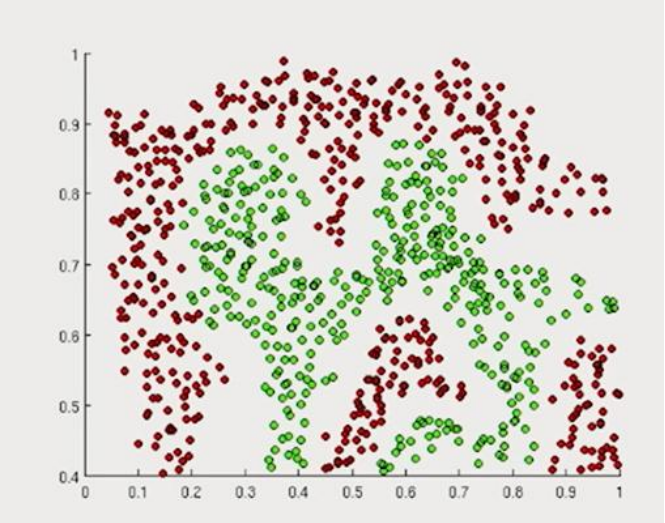
$$\hat{y} = g(w_0 + \boldsymbol{X}^T \boldsymbol{W})$$

where: $\boldsymbol{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$ and $\boldsymbol{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$

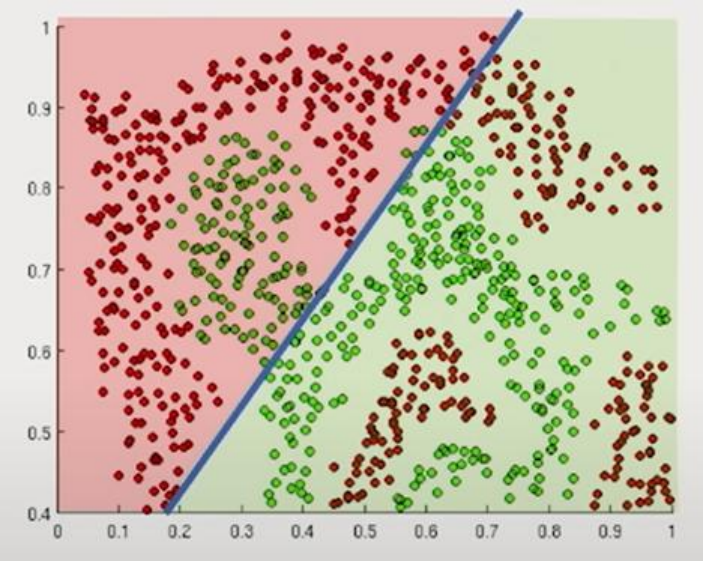
UNIVERSITY OF SUSSEX

Why do we need Activation Functions?

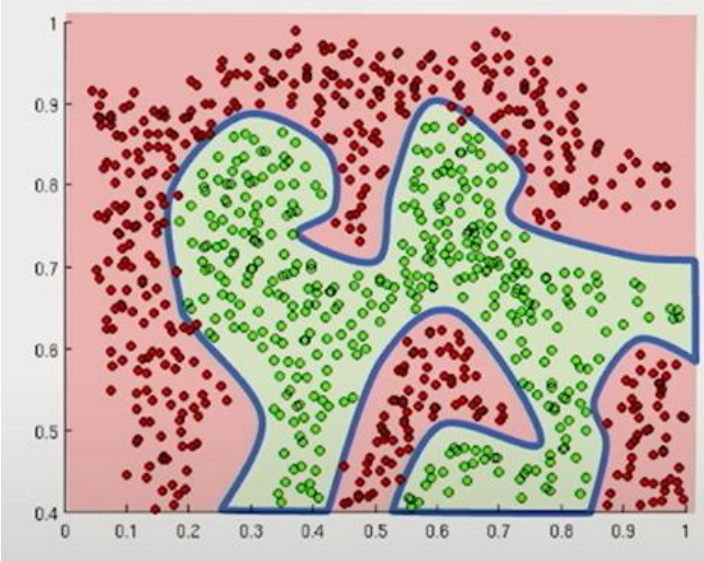
The purpose of activation functions is to introduce **non-linearities** into the network



Why do we need non-linearity?



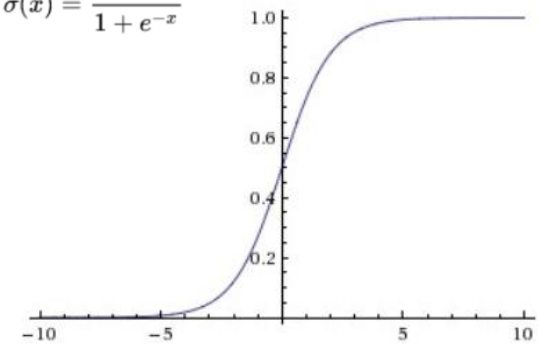
Linear activation functions produce **linear decisions** no matter the network size



Non-linearities allow us to **approximate** arbitrarily complex functions

What types of activation functions are there and how do we pick one?

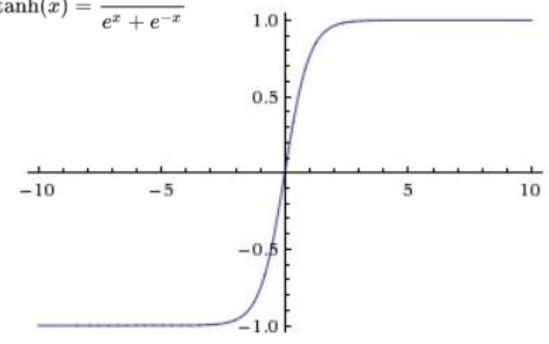
$f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$



Sigmoid non-linearity squashes the numbers to range between [0,1].

Used for models where we have to predict the probability as an output

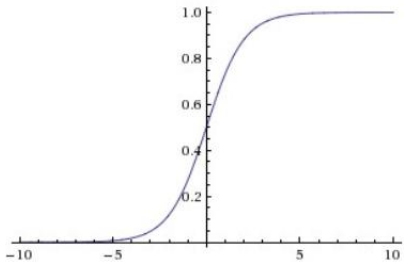
$f(x) = \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$



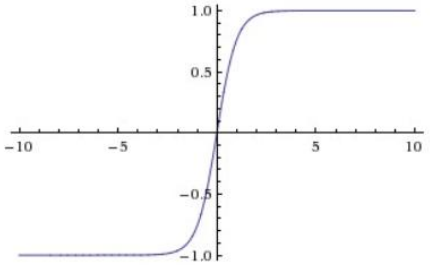
tanh non-linearity squashes the numbers to range between [-1,1]

Tanh tends to be preferred to sigmoid

What types of activation functions are there and how do we pick one?

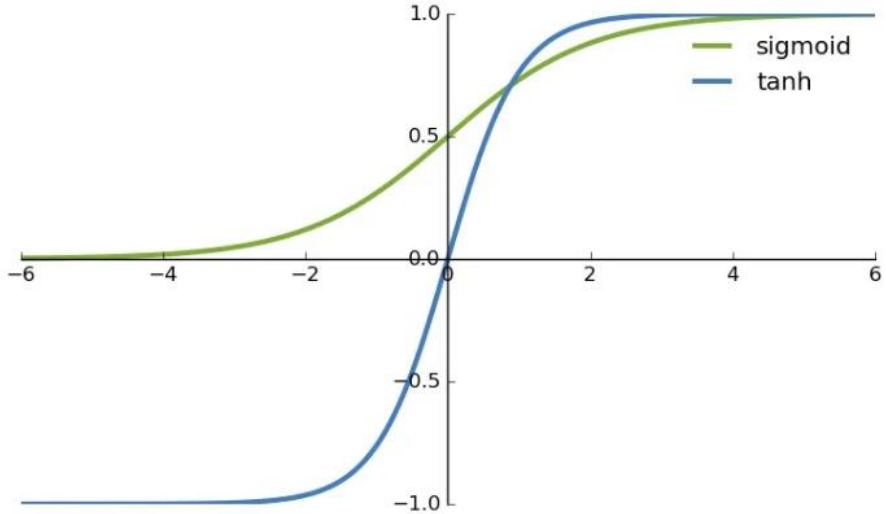



```python
from sklearn.neural_network import MLPClassifier

# Define the neural network with the sigmoid activation
function
clf = MLPClassifier(activation='logistic', hidden_layer_sizes=
(100,), max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)
```




```python
from sklearn.neural_network import MLPClassifier

# Define the neural network with the tanh activation function
clf = MLPClassifier(activation='tanh', hidden_layer_sizes=
(100,), max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)
```

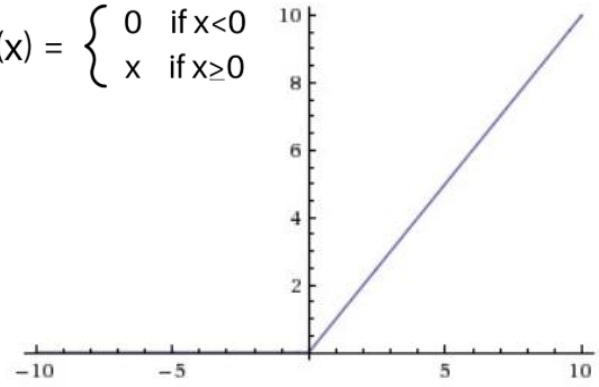
What types of activation functions are there and how do we pick one?



What types of activation functions are there and how do we pick one?

f(x)=max(0,x)

$$f(x) = \begin{cases} 0 & \text{if } x<0 \\ x & \text{if } x \geq 0 \end{cases}$$

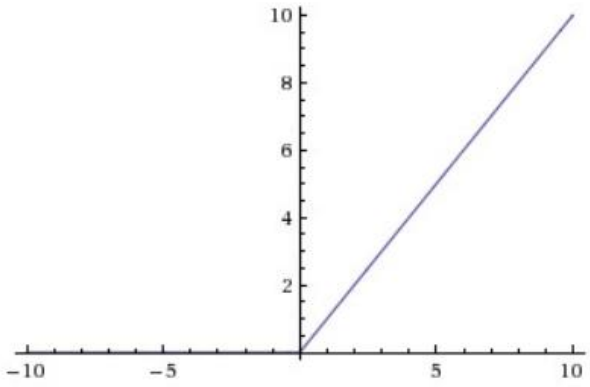


ReLu (Rectified Linear Unit) activation function is zero when x < 0 and then linear with slope 1 when x > 0

- **ReLu speeds up** the calculations up to 6x compared to **tanh**
- **ReLU units** can be fragile during training and can **"die"**. (i.e. neurons that never activate across the entire training dataset) if the learning rate is set too high. **With a proper setting of the learning rate this is less frequently an issue.**

What types of activation functions are there and how do we pick one?

f(x)=max(0,x)

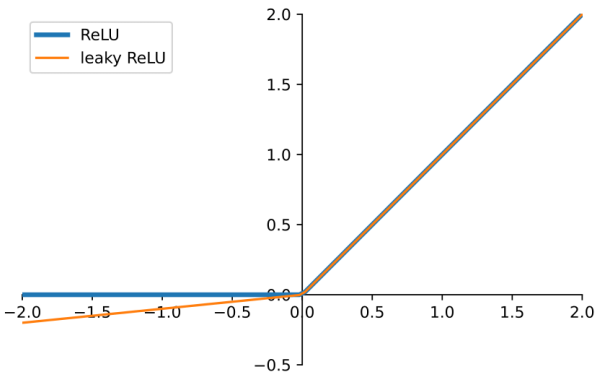


ReLu (Rectified Linear Unit) activation function is zero when x < 0 and then linear with slope 1 when x > 0

```python
from sklearn.neural_network import MLPClassifier

# Define the neural network with the ReLU activation function
clf = MLPClassifier(activation='relu', hidden_layer_sizes=(100,), max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)
```

What types of activation functions are there and how do we pick one?

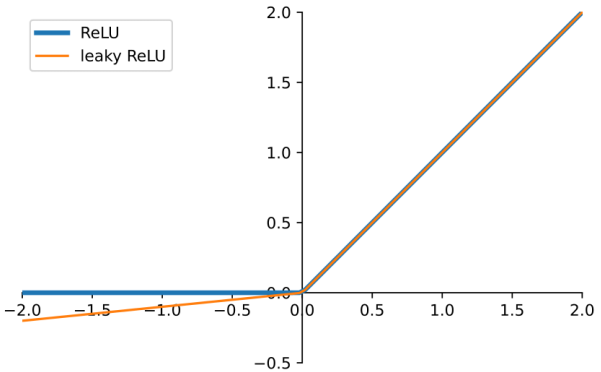
$$f(x) = \begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



- **Leaky ReLUs** are one attempt to fix the "dying ReLU" problem**.**
- Instead of the function being zero when x < 0, a leaky ReLU will instead have a small positive slope (of 0.01, or so). That is, the function computes **f(x)=1(x<0)(αx)+1(x>=0)(x)** where **α** is a small constant**.**

Other solutions to the "dying ReLU" problem include ELUs (Exponential Linear Units) and PReLUs (Parametric ReLU).

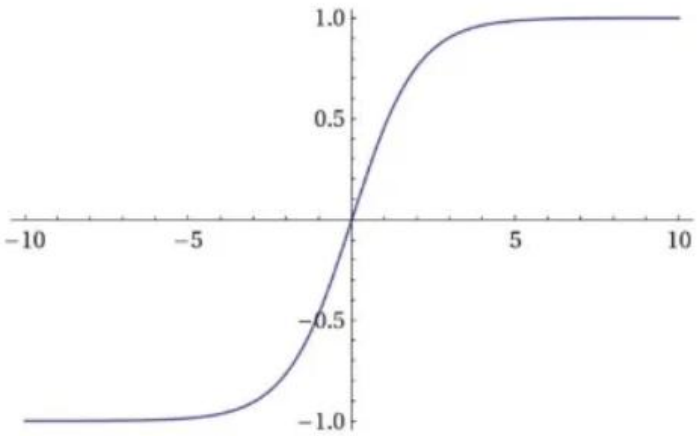
What types of activation functions are there and how do we pick one?



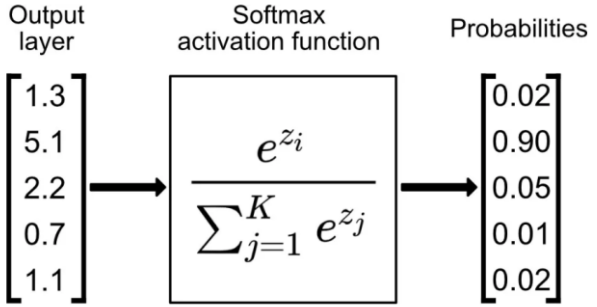
- **Leaky ReLUs** are one attempt to fix the “dying ReLU” problem.

```python
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LeakyReLU

# Define the neural network model
model = Sequential([
    # Add a dense layer with 100 neurons
    Dense(100, input_dim=input_dim),
    # Apply Leaky ReLU activation function with alpha=0.01
    LeakyReLU(alpha=0.01),
    # Add the output layer with softmax activation for multi-class classification
    Dense(output_dim, activation='softmax')
])

# Compile the model with Adam optimizer and categorical crossentropy loss
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the neural network on the training data
model.fit(X_train, y_train, epochs=300)
```

What types of activation functions are there and how do we pick one?

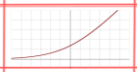
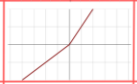
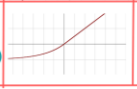


- **Softmax** is used when you need to classify data into more than two categories. For example, if you have a dataset with images of cats, dogs, and birds, Softmax can help determine the probability that a given image belongs to each of these classes.
- It is applied after the output.



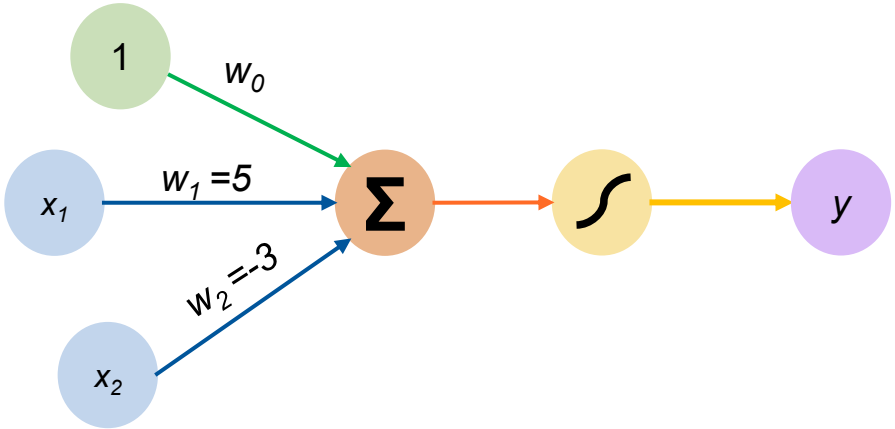
UNIVERSITY OF SUSSEX

What types of activation functions are there and how do we pick one?

ACTIVATION FUNCTION	PLOT	EQUATION	DERIVATIVE	RANGE
Linear		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Binary Step		$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	{0, 1}
Sigmoid		$f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$	$f'(x) = f(x)(1 - f(x))$	(0, 1)
Hyperbolic Tangent(tanh)		$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$	$f'(x) = 1 - f(x)^2$	(-1, 1)
Rectified Linear Unit(ReLU)		$f(x) = \begin{cases} 0 & \text{if } x<0 \\ x & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	[0, ∞)
Softplus		$f(x) = \ln(1 + e^{x})$	$f'(x) = \frac{1}{1+e^{-x}}$	(0, 1)
Leaky ReLU		$f(x) = \begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	(-1, 1)
Exponential Linear Unit(ELU)		$f(x) = \begin{cases} \alpha(e^{x} - 1) & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$	$f'(x) = \begin{cases} \alpha e^{x} & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ 1 & \text{if } x = 0 \text{ and } \alpha = 1 \end{cases}$	[0, ∞)

UNIVERSITY OF SUSSEX

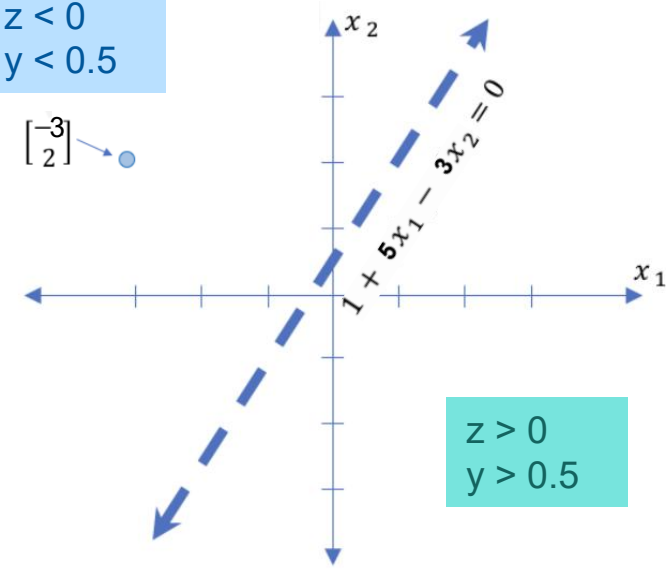
Perceptron: an example



$$\hat{y} = g(1 + 5x_1 - 3x_2)$$

(z)

z < 0
y < 0.5



z > 0
y > 0.5

If we have an input $X = \begin{bmatrix} -3 \\ 2 \end{bmatrix}$

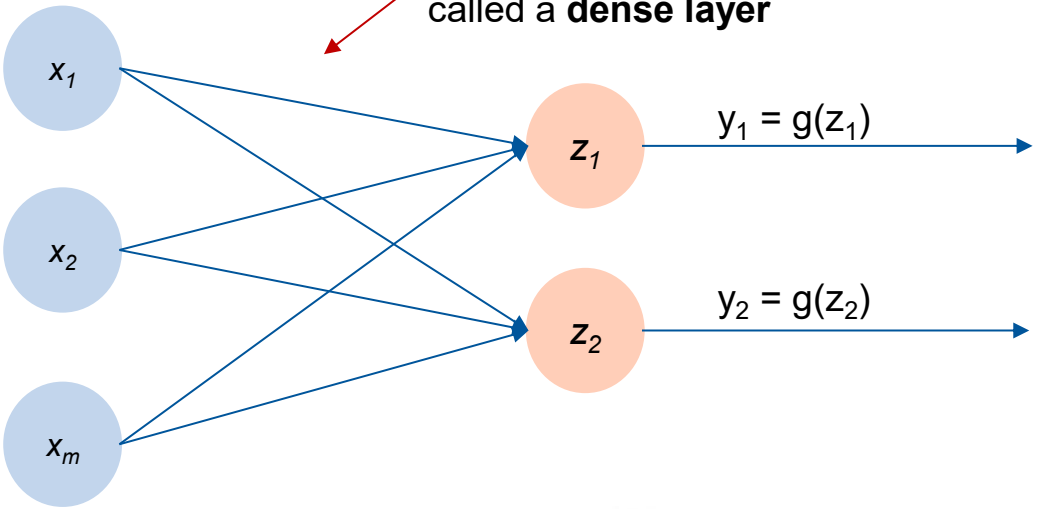
$$\hat{y} = g(1 + (5 * -3) - (3 * 2))$$

$$= g(-20) = 2.06 \text{ E-9}$$

UNIVERSITY OF SUSSEX

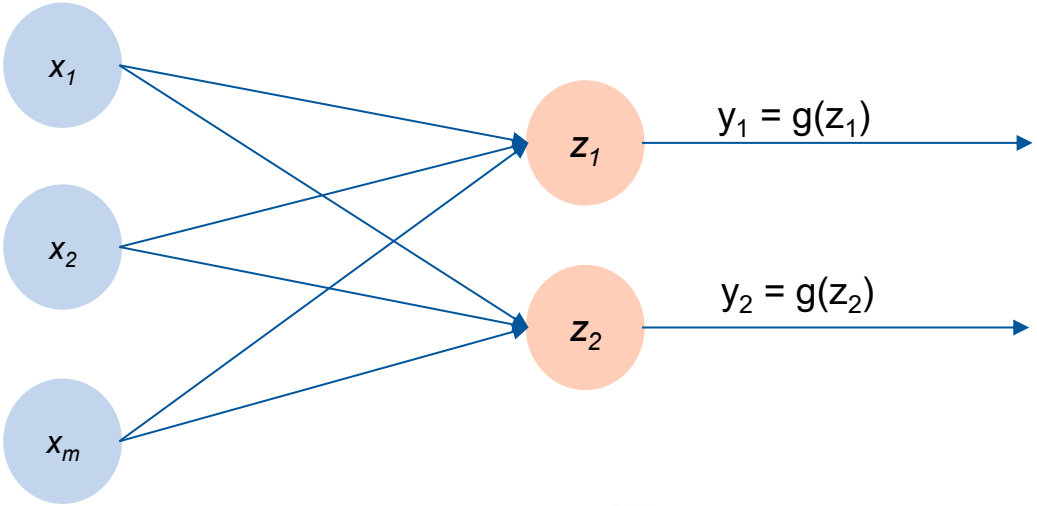
Multi-output perceptron

Because of the dense connections between inputs and outputs, this is called a **dense layer**



$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

Multi-output perceptron

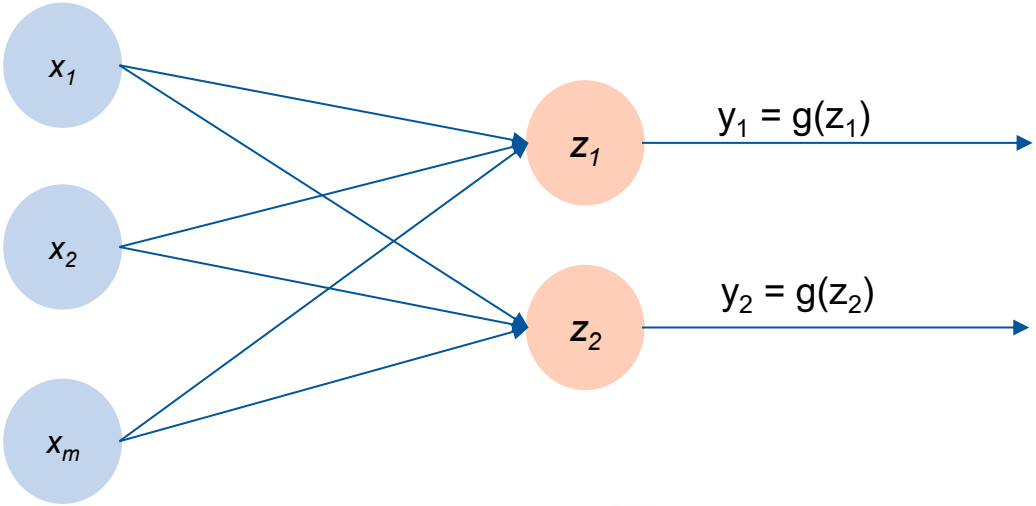


This is just an example to understand the intuition.

Multi-output perceptrons are useful when you need to predict multiple target variables simultaneously.

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

Multi-output perceptron



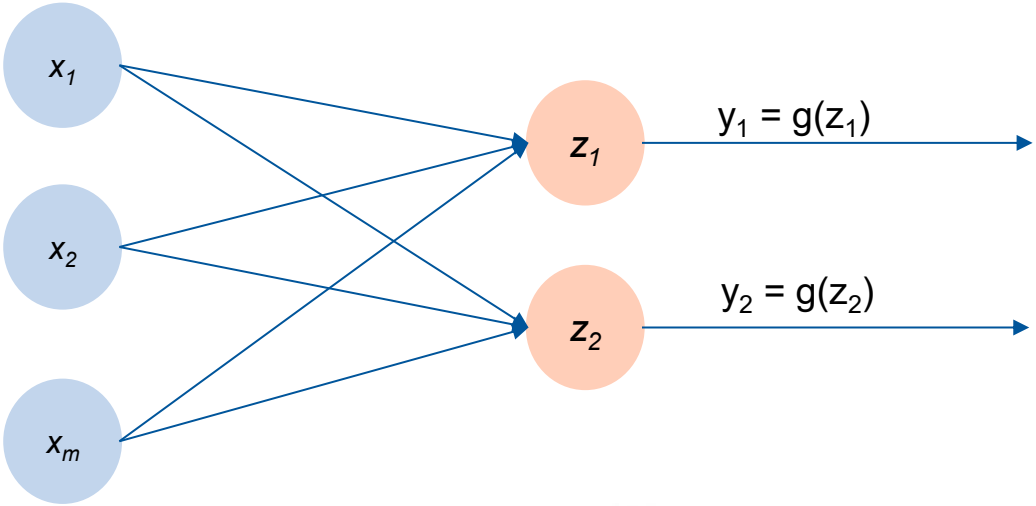
NEURAL DECODING

By inputting population neural activity (e.g. calcium imaging/spike trains) and output motor variables, decision outcomes, and predicted reward.

Multi-output advantage: Simultaneous decoding of multiple behavioural signals components from the same neural state.

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

Multi-output perceptron



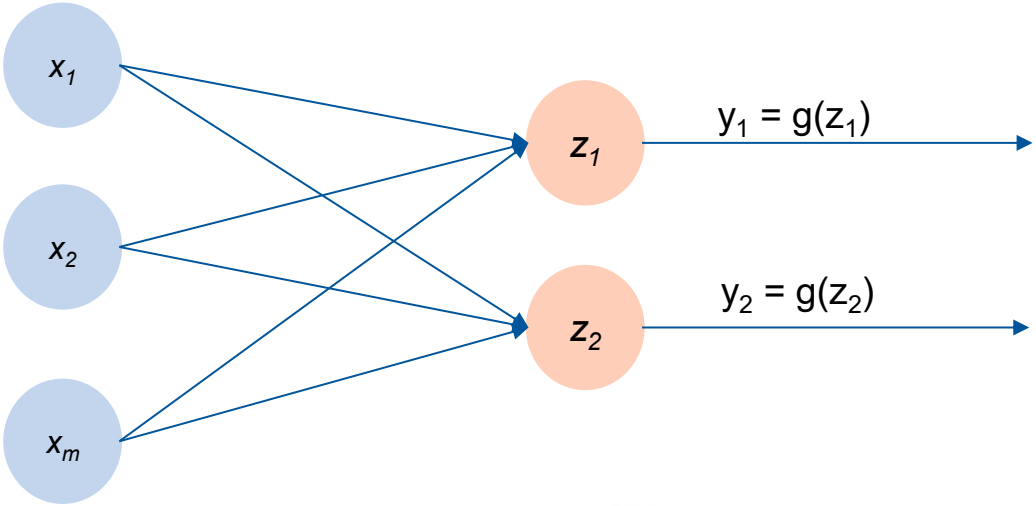
STIMULUS ENCODING

Like predicting neural responses to external stimuli, by inputting auditory/visual stimuli (e.g. frequency, orientation, temporal feature), and output the activity of multiple neurons.

Multi-output advantage: The network learns a population code, capturing how a group of neurons responds in parallel.

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

Multi-output perceptron



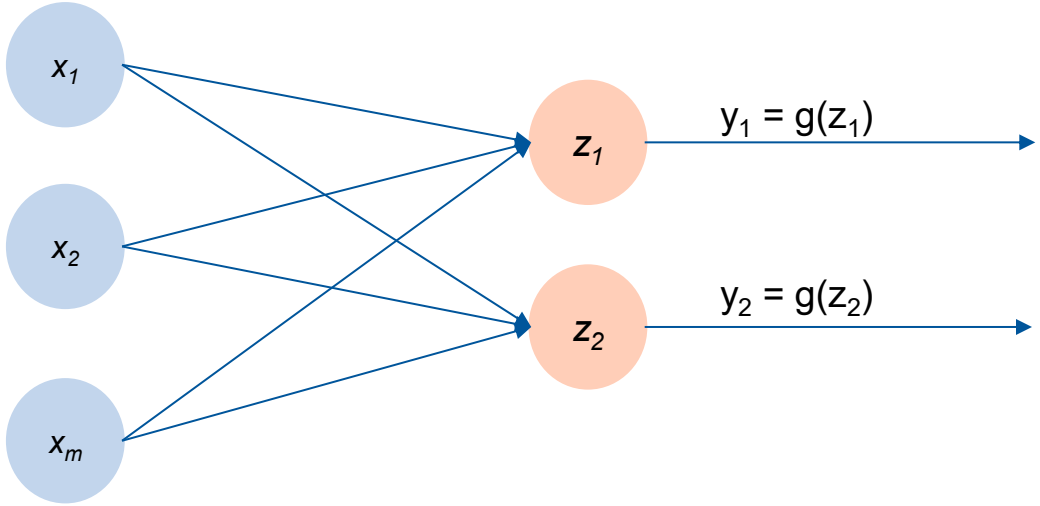
BEHAVIOURAL CLASSIFICATION WITH RICH LABELS

Like classifying trial types or internal states with multidimensional labels by predicting not only “lick” or “no lick” but also trial phase (stimulus, response), stimulus identity, etc.

Multi-output advantage: jointly learn latent task variables that may be entangled in the neural data.

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

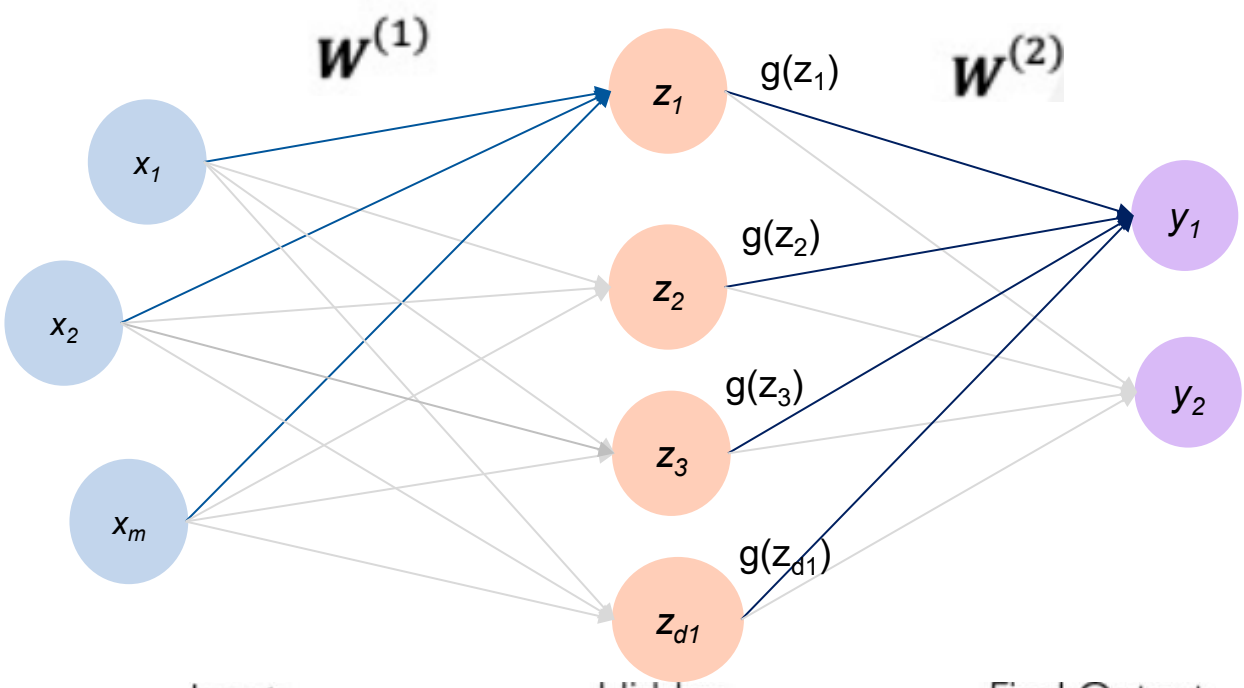
Multi-output perceptron




```python
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# Generate synthetic data with multiple outputs
X, y = make_regression(n_samples=1000, n_features=20,
n_targets=2, noise=0.1)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Define the multi-output perceptron
mlp = MLPRegressor(hidden_layer_sizes=(100,),
activation='relu', max_iter=300)

# Train the neural network on the training data
mlp.fit(X_train, y_train)

# Make predictions on the test data
predictions = mlp.predict(X_test)
```

$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

Single Layer Neural Network



In layered neural networks, the outputs from one layer become the inputs of the next layer

$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^{m} x_j \, w_{j,i}^{(1)} \qquad \hat{y}_i = g\left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) \, w_{j,i}^{(2)}\right)$$

UNIVERSITY OF SUSSEX

Single Layer Neural Network

For classification:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate synthetic classification data
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2,
random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Define the single-layer neural network
clf = MLPClassifier(hidden_layer_sizes=(100,), activation='relu',
max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)

# Make predictions on the test data
predictions = clf.predict(X_test)
```

For regression:

```python
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# Generate synthetic regression data
X, y = make_regression(n_samples=1000, n_features=20, noise=0.1,
random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Define the single-layer neural network
reg = MLPRegressor(hidden_layer_sizes=(100,), activation='relu',
max_iter=300)

# Train the neural network on the training data
reg.fit(X_train, y_train)

# Make predictions on the test data
predictions = reg.predict(X_test)
```

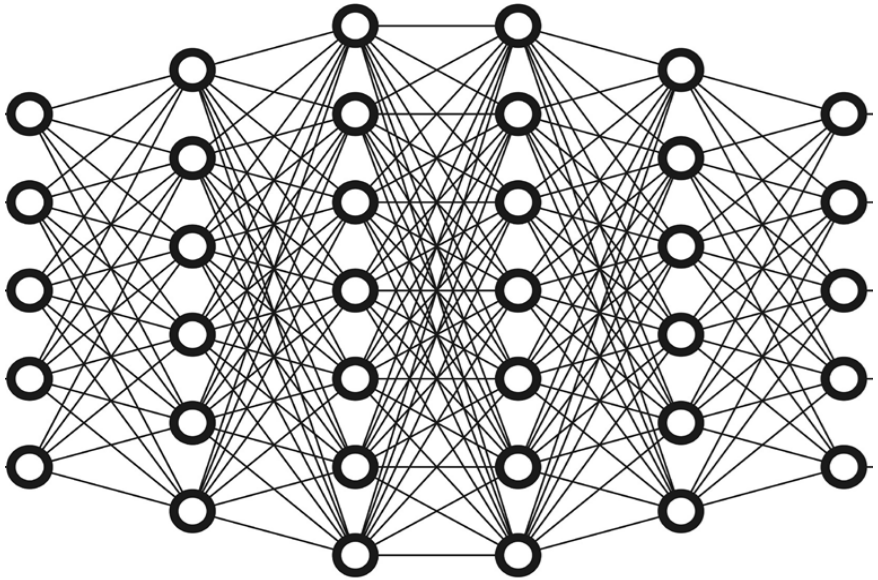
Single Layer Neural Network

For classification:

```python
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate synthetic classification data
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2,
random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Define the single-layer neural network
clf = MLPClassifier(hidden_layer_sizes=(100,), activation='relu',
max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)

# Make predictions on the test data
predictions = clf.predict(X_test)
```

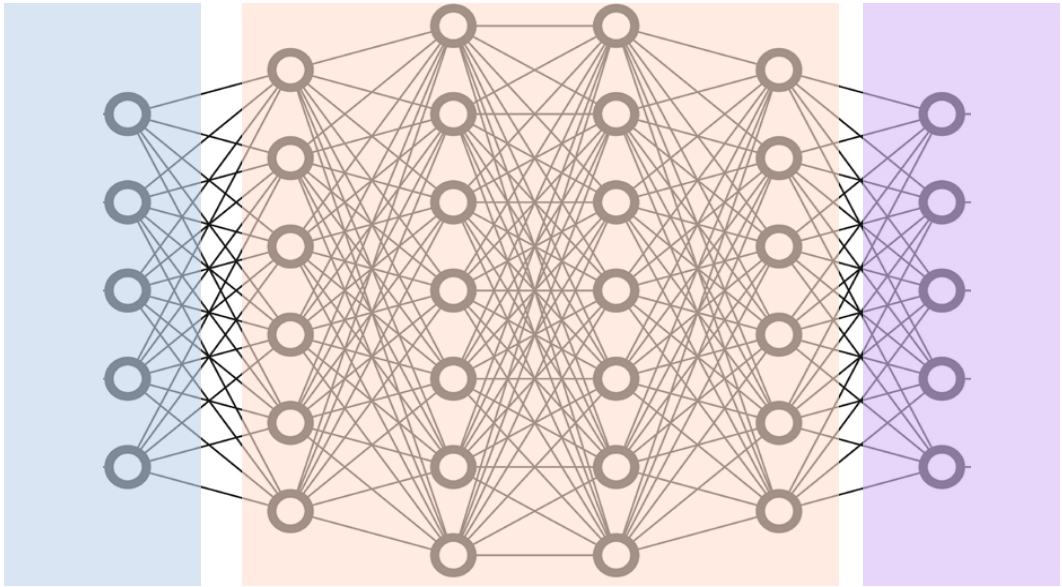
- **n_samples**= The number of samples (data points) in your dataset. 1000 creates 1000 data points.
- **n_features**=The number of features (input variables) in your dataset. 20 means each data point has 20 input variables.
- **n_classes**= It determines how many different categories the target variable (y) can take.2 means the target variable (y) will have three distinct class labels (e.g., 0, 1).
- **random_state**=A seed value for random number generation to ensure reproducibility. 42 ensures the data generation process is reproducible.

Multi Layer Neural Network



In layered neural networks, the outputs from one layer become the inputs of the next layer

Multi Layer Neural Network



Inputs

Hidden Layers

Outputs

In layered neural networks, the outputs from one layer become the inputs of the next layer

Multi Layer Neural Network

For classification:

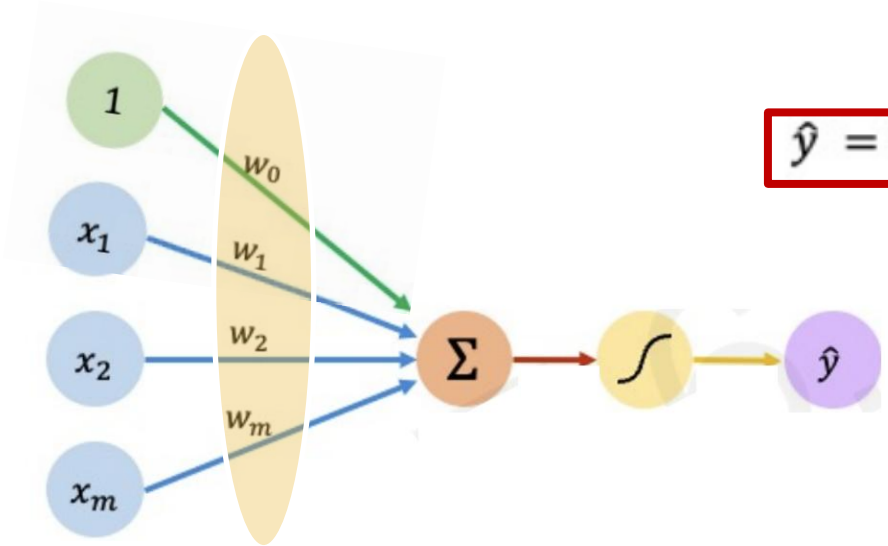
```python
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate synthetic classification data
# n_samples: number of samples
# n_features: number of features
# n_classes: number of classes
# random_state: seed for reproducibility
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split the data into training and testing sets
# test_size: proportion of the dataset to include in the test split
# random_state: seed for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the multi-layer neural network with two hidden layers
# hidden_layer_sizes: tuple specifying the number of neurons in each hidden layer
# activation: activation function for the hidden layers ('relu' in this case)
# max_iter: maximum number of iterations for training
clf = MLPClassifier(hidden_layer_sizes=(100, 50), activation='relu', max_iter=300)

# Train the neural network on the training data
# X_train: training input data
# y_train: training target data
clf.fit(X_train, y_train)

# Make predictions on the test data
# X_test: testing input data
predictions = clf.predict(X_test)
```

- **hidden_layer_sizes**= in this case it specifies two hidden layers, the first with 100 neurons and the second with 50 neurons.

Training a neural network

The code does it for us. But what does it do exactly?

Think about your data and think about this model



$$\hat{y} = g(w_0 + \boldsymbol{X}^T \boldsymbol{W})$$

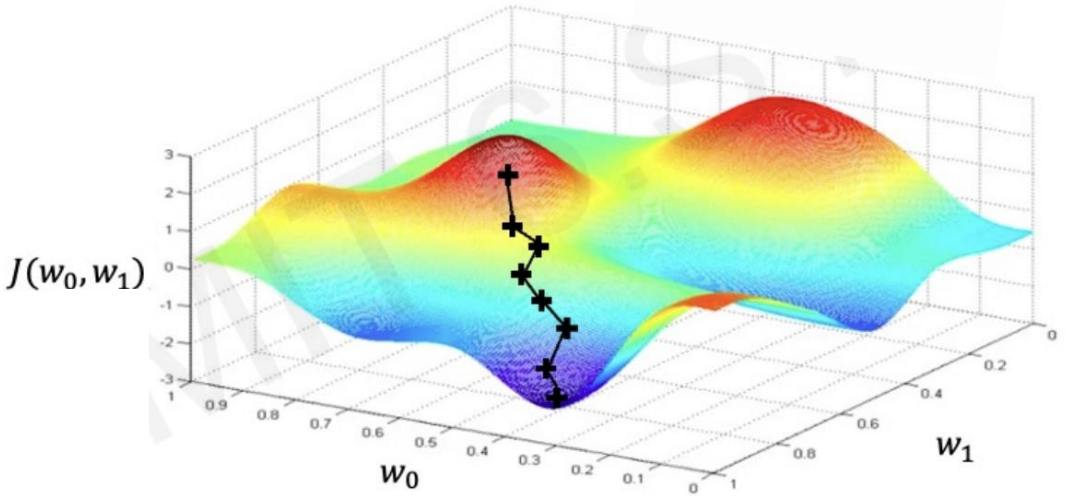
Is there any information we don't have?

Loss optimisation

- Quantify the Loss (how wrong the outputs y are compared to our test data)
- Update the weights (through gradient descent and backpropagation) to minimise this loss

Loss optimisation

- Quantify the Loss (how wrong the outputs y are compared to our test data)
- Update the weights (through gradient descent and backpropagation) to minimise this loss



scikit-learn handles all the underlying details of backpropagation and gradient descent, allowing you to focus on defining and training your model

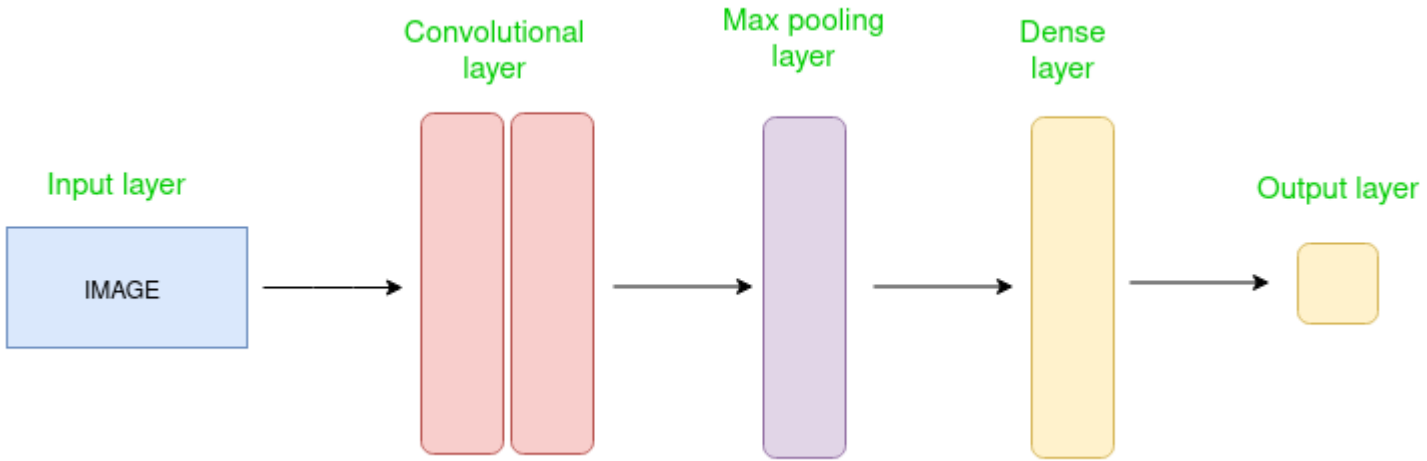
The code handles this for us

Types of ANNs

Convolutional Neural Networks (CNNs)

CNNs are designed for visual data processing.

Convolutional neural networks

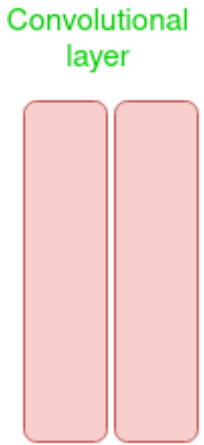


Convolutional neural networks

Let’s take an example by running an image of dimension 32 x 32 x 3.

- **Input Layers:** It’s the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3 (= the colour array).

Convolutional neural networks



- **Filters/Kernels**: Small matrices that slide over the input image. *Each filter is designed to detect specific features like edges, textures, or patterns.*
- **Convolution Operation**: The filter is applied to the input image, producing a feature map. This operation involves multiplying the filter values with the corresponding pixel values and summing them up. The result is a new matrix that highlights the presence of the feature detected by the filter. *As the window slides over the image, it creates a new picture (feature map) that highlights where it found those edges or patterns.*
- **ReLU:** After the window slides over the image, ReLU changes all negative values to zero. *This helps the network focus on important features and ignore less useful information.*

Convolutional neural networks

- **Max Pooling**: This layer reduces the dimensionality of the feature map while retaining important features. It does this by selecting the maximum value from a small region (e.g., 2x2) of the feature map. This helps in reducing computational load and preventing overfitting.
- *Think of s zooming out on a picture to make it smaller while keeping the important parts. Max pooling picks the biggest value from a small area, reducing the size of the feature map but keeping the key features.*

Convolutional neural networks

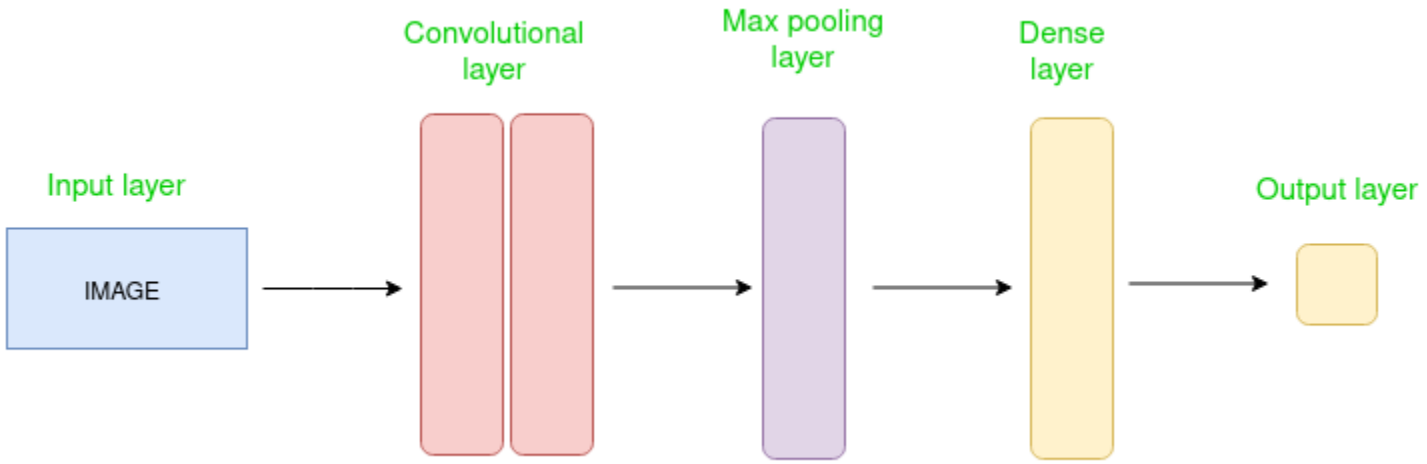
- **Flattening:** Converts the 2D feature maps into a 1D vector. This step prepares the data for the fully connected layers. *Imagine taking all the important features from the picture and laying them out in a single line.*

- **Dense Layers:** These layers connect every neuron from one layer to every neuron in the next layer. They combine the features learned by the convolutional and pooling layers to make final predictions. ***This is where the network decides which class the input image belongs to.***

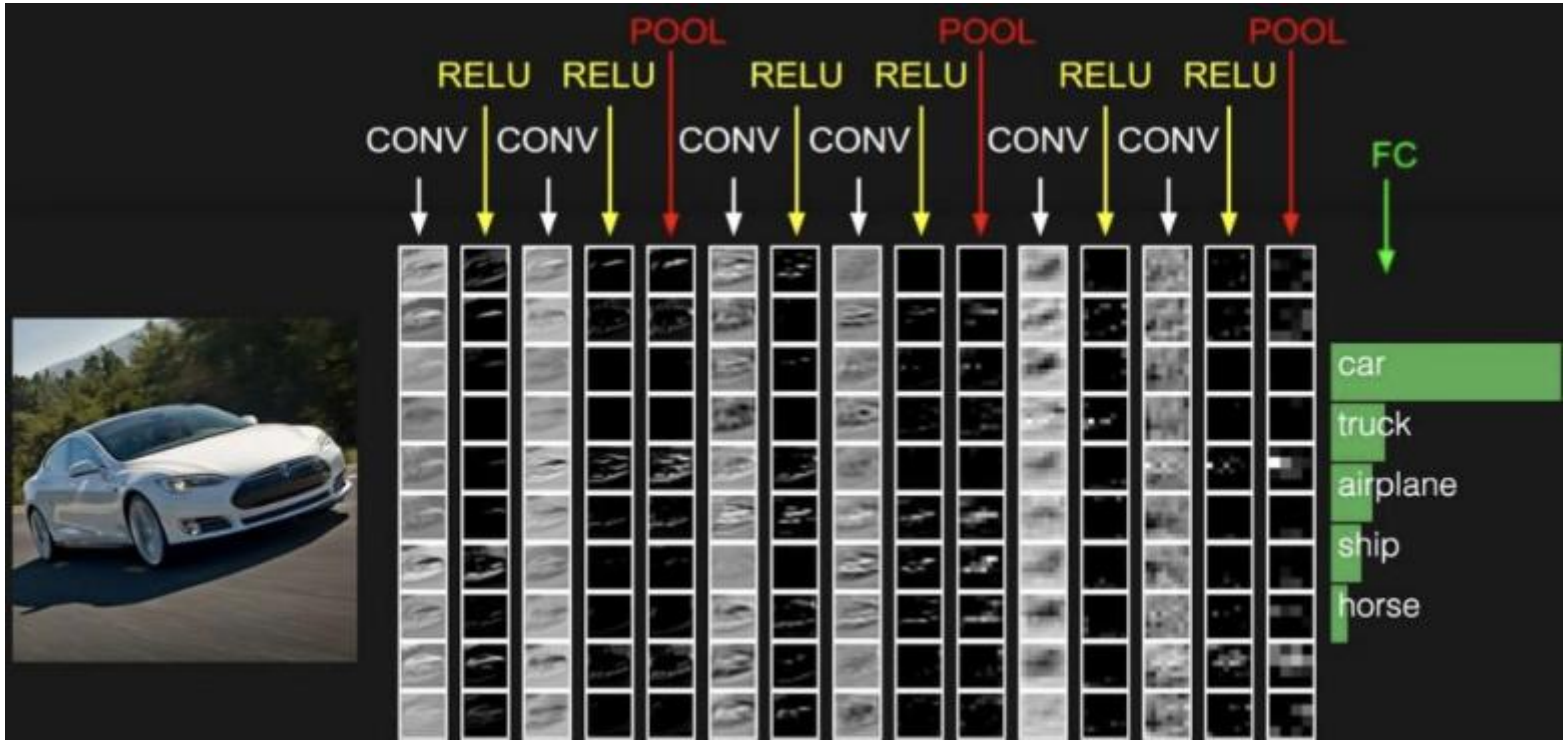
Convolutional neural networks

- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like *sigmoid* or *softmax* which converts the *output of each class into the probability score of each class*

Convolutional neural networks



Convolutional neural networks

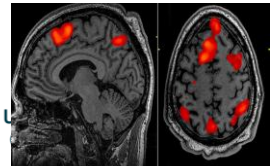


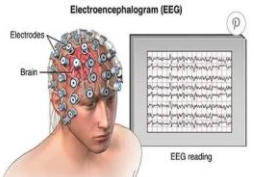
UNIVERSITY OF SUSSEX

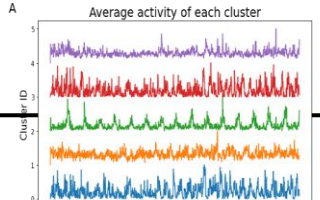
Convolutional neural networks in neuroscience

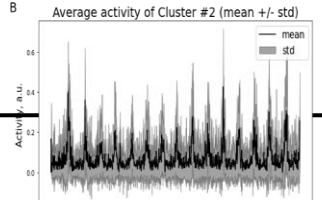
- **Brain Imaging:**
  - Structural MRI/fMRI: Disease diagnosis, brain age prediction
  - Calcium imaging: Spike deconvolution, neuron segmentation
- **Signal Processing:**
  - Spike sorting from extracellular recordings
  - EEG/LFP decoding of brain states
- **Behavioural Analysis:**
  - Pose estimation and action recognition from video (e.g., DeepLabCut)
- **Computational Neuroscience:**
  - Modelling visual cortex responses to natural stimuli
  - Feature extraction and representational similarity analysis

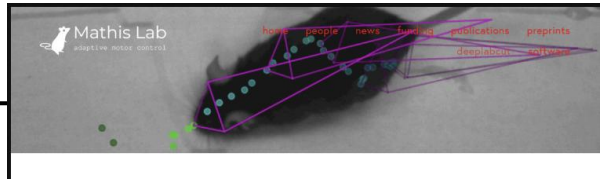
Image Suggestion: Grid of application icons (brain scan, electrode trace, video frame, CNN-visual cortex overlay)











Convolutional neural networks

```python
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Define the CNN model
model = Sequential()

# Add a convolutional layer with 32 filters, a 3x3 kernel, and ReLU activation
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64, 3)))

# Add a max pooling layer with a 2x2 pool size
model.add(MaxPooling2D(pool_size=(2, 2)))

# Add another convolutional layer with 64 filters
model.add(Conv2D(64, (3, 3), activation='relu'))

# Add another max pooling layer
model.add(MaxPooling2D(pool_size=(2, 2)))

# Flatten the feature maps to a 1D vector
model.add(Flatten())

# Add a fully connected layer with 128 neurons and ReLU activation
model.add(Dense(128, activation='relu'))

# Add the output layer with softmax activation for multi-class classification
model.add(Dense(10, activation='softmax'))

# Compile the model with Adam optimizer and categorical crossentropy loss
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Print the model summary
model.summary()"
```

1.Convolutional Layers:

1. **Number of Filters**: The choice of 32 and 64 filters is typical for initial layers in a CNN. These numbers can be adjusted based on the complexity of the task and the size of the dataset.
2. **Kernel Size**: A 3x3 kernel is a standard choice that balances computational efficiency and the ability to capture spatial features.

2.Pooling Layers:

1. **Pool Size**: A 2x2 pool size is commonly used to reduce the spatial dimensions while retaining important features.

3.Fully Connected Layers:

1. **Number of Neurons**: The choice of 128 neurons in the dense layer is a common practice, but it can be tuned based on the complexity of the task.

4.Activation Functions:

1. **ReLU**: The ReLU activation function is widely used due to its ability to mitigate the vanishing gradient problem and speed up training.
2. **Softmax**: Softmax is used in the output layer for multi-class classification tasks.

5.Optimizer and Loss Function:

1. **Adam Optimizer**: Adam is a popular choice for its adaptive learning rate and efficiency.
2. **Categorical Crossentropy**: This loss function is suitable for multi-class classification problems.

UNIVERSITY OF SUSSEX

[Diving Deep into CNNs: Optimization and Regularization - Neural Nexus]

UNIVERSITY
OF SUSSEX